Cluster Computing (2025) 28:330
https://doi.org/10.1007/s10586-024-04992-5

Drone-as-a-Service: proximity-aware composition of UAV-based delivery services

Mokhtar Sellami[1] · Haithem Mezni[2] · Hela Elmannai[3] · Reem Alkanhel[3]

Received: 6 September 2024 / Revised: 1 November 2024 / Accepted: 8 December 2024 / Published online: 28 April 2025

Abstract
Recently, drones have emerged as an effective and rapid mean to deliver various packages as services. Exhibiting the same behavior and principles of the service paradigm, drones' delivery tasks can be abstracted as services, also called DaaS (Drone-as-a-Service). However, drones operate in a dynamic and uncertain environment (flight regulations, weather conditions, charging resources). In addition, the drones' limited capacity (e.g., flying range, battery capacity, payload) and the delivery constraints (e.g., cost, delivery time) make the delivery missions beyond one drone's capacity. To cope with the above challenges, and to meet the customers' and providers' delivery constraints, composing drone services is a natural solution. We propose a DaaS composition approach that takes advantage of a recent and powerful technology, called knowledge graph. First, we model the drones' environment (charging stations, skyway segments, drone services) as a heterogeneous information network. Then, we combine knowledge graph embedding with subgraph-aware proximity measure, to arrange drones and charging stations in a low-dimentional vector space. This pre-processing step helped select high-flight capacity drone services, and generate low-congestion delivery paths. Experimental results have proven the approach's performance and the quality of DaaS compositions, compared to recent state-of-the-art approaches.

Keywords Drone as a Service · DaaS composition · Knowledge graph · Metapath-based embedding · Proximity-aware selection

1 Introduction

Nowadays, drones are used in commercial and private sectors to cover a wide range of requirements, including aerial photography and videography, items delivery, surveillance, continuous forecasting, flight operation management, training and education, utility inspections, search and rescue missions, etc. [1]. Delivery drones are autonomous Unmanned Aerial Vehicles (UAV), used in the transportation of various packages (e.g., food, medical products, pharmaceuticals samples) in commercial, industrial and medical sectors [1]. The use of drones to deliver packages was first announced by Amazon in 2013. Since then, leading companies like Google in 2014 and NASA in 2015 announced their drone projects. Popular delivery solutions include Amazon's Prime Air drone, Google's Wing drones, HEMAV for drone services in agriculture, Zipline delivery drone services to carry emergency supplies to disaster zones, Cyberhawk's inspection drones, etc. Other providers (e.g., Dronegenuity, Fly Guys, SkyNet, DroneBase) offer drones as services by renting them to companies.

Drone-based delivery services, also called *Drone as a Service* (DaaS) in [2], take advantage of the service computing paradigm [3], which abstracts the drones' functional (e.g., flight missions) and non-functional (e.g., flight

✉ Hela Elmannai
hselmannai@pnu.edu.sa

Mokhtar Sellami
mokhtar.sellami@fsjegj.rnu.tn

Haithem Mezni
hmezni@taibahu.edu.sa

Reem Alkanhel
rialkanhal@pnu.edu.sa

[1] Jendouba University, Jendouba, Tunisia

[2] Taibah University, Madinah, Saudi Arabia

[3] Department of Information Technology, College of Computer and Information Sciences, Princess Nourah bint Abdulrahman University, Riyadh 11671, Saudi Arabia

capacities) behavior as services. DaaS functional properties concerns the delivery of packages following a skyway path (a set of skyway segments). Non-functional properties include battery capacity, payload, flight range, speed, etc. [4]. The skyway network consists of a set of nodes that can be recharging stations or pickup/delivery locations. The drones' limited capacity and the dynamic nature of their sky network (e.g., weather conditions, stations availability) influence their flight range [5]. To cope with these intrinsic and extrinsic factors [6] and to achieve their missions, drones must cooperate (e.g., swarming drones) and recharge their battery multiple times at different charging stations. As a promising solution to these issues, DaaS composition has emerged to become an effective means of combining several delivery services, together with their skyway segments traversed from a source to a target location [2].

Initially treated as a Vehicle Routing Problem (VRP-D) and a Travel Salesman Problem (TSP-D), drone services' composition has been tackled by few researchers (e.g., [2, 4, 7–16]), due to its novelty. The few attempts' main goal was to find an optimal subset of skyway segments for the drone(s) selected to fulfil the delivery tasks, while minimizing the delivery cost and time, as well as the drones' energy consumption. However, the problem of DaaS composition was treated as a traditional shortest path search. Add to that, current solutions lack a complete and semantic modeling of the sky network's major entities (e.g., drone services, charging stations, skyway segments), which may delay the consumer's acceptance of drone services [1]. To cope with these issues, this paper aims to provide a rich modeling of the delivery drones' network. We also incorporate several DaaS-related and environmental constraints into the DaaS composition, to ensure the scheduling of time-effective and low-cost delivery services. That is by combining several DaaSs across multiple stations, while taking these latter's proximity and charging capacities into consideration.

Our main contributions are summarized as follows:

- We model the drones' network as a semantic multi-relational knowledge graph, called *DaaSKG*.
- We use knowledge graph embedding (KGE) [17] to transform the DaaSKG into a low-dimensional vector space, facilitating hence its processing and exploration.
- We define a proximity-aware method that allows composing the drone services (with high flight/delivery capacity) involved in the delivery mission.
- We implement and evaluate our approach using a real-world drones dataset, and we compare it with three DaaS composition approaches.

This paper is organized as follows: Section 2 summarizes the current solutions to the DaaS composition problem. Section 3 formulates the DaaS composition problem. In Sections 4 and 5, we propose a knowledge graph modeling and embedding of the DaaS network, and we present the proximity-aware DaaS composition process. Section 6 provides a theoretical study of the approach's complexity. Section 7 discusses the experimental results. Finally, Section 8 is devoted to the conclusion and the future work.

2 Related work

The problem of drone-based package delivery has been initially treated from a scheduling and routing perspective [18]. Existing researches have formulated the drones' scheduling and routing as a *Travelling Salesman Problem* (TSP-D) [19–30] (i.e. single drone) or a *Vehicle Routing Problem* (VRP-D) [5, 31–43] (multiple drones). Recent attempts have applied the service computing paradigm [3] to abstract the drone-based delivery tasks as services, also called *Drone-as-a-Service* (DaaS) [2, 4, 8, 9, 11–16].

Regardless of their design orientations, researchers have targeted different *goals* (e.g., last-mile delivery, return, simultaneous pickup and delivery), *constraints* (e.g., travel cost, conflicts, battery budget, total profit, weather conditions, payload), and *techniques* (e.g., simulated annealing [44], variable neighborhood descent algorithm [44], approximation algorithms [45]). We give as examples, last-mile delivery [46] with mixed-integer linear programming, heterogeneous drone delivery with multi-objective optimization [47] and for multiple consumers [12], multiple drone delivery scheduling with a fixed number of drones [45], multiple single depot scheduling [44], etc.

2.1 TSP-based drone routing approaches

In this class, some approaches [22–27] deal with the TSP-D by incorporating different constraints, such as transportation cost minimization [24], drone routes' optimization [25], risk and make-span in contactless delivery [30]. Other approaches [19] have tackled the parallel scheduling of trucks' and drones' missions to ensure an independent and parallel package delivery. TSP-D approaches apply techniques like Simulated Annealing [44], Mixed Integer Programming (MIP) [23, 25, 27, 48], Greedy Randomized Adaptive Search [24], k-means clustering [21], Genetic Algorithm [21], and dynamic programming [26]. Mathematical programming [19], optimization [21, 25], and Fitness-Distance Balance-based evolutionary search [49] have also been employed to formulate and find the optimal delivery paths, or to optimize Drone, truck routing [49].

Some attempts [19, 28] have focused on the restrictions of flying areas and the effects of payload on drones' energy consumption [28], while others (e.g., Flying Sidekicks TSP

[19]) have concentrated their efforts on the coordination of a single traditional delivery truck with a single drone. To cope with the complexity of specific delivery tasks, some researchers recommend combining multiple trucks and drones in response to heterogeneous delivery problems. For example, Ferrandez et al. [21] combined k-means and Genetic Algorithm to propose an optimization model that allows finding the most efficient launch locations in a Drone, truck delivery network, and to assign the truck route between those launch locations.

The TSP-D problem has been extended in [20, 29, 48] to consider multiple drones and trucks. For example, Kitjacharoenchai et al. [29] propose to retrieve drones by any nearby truck to achieve the delivery task. Murray and Raj [48] propose to deploy an arbitrary number of heterogeneous UAVs from the depot or the delivery truck. However, the above approaches do not provide rich models to describe the drones environment, and ignore important constraints (drones capacities, charging stations' resources) that may affect the quality of delivery tasks.

2.2 VRP-based drone routing approaches

The solutions, in this class (VRP with Drones), instantiate the vehicle routing problem to solve different drone routing issues [5, 31–41]. Researchers have incorporated different constraints and parameters, such as the impact of drones' speed and number on the energy saving [31], the effects of drones' battery capacity, cost metrics, and drones' deployment cost on time saving [32], the incorporation of time windows in the drones' routing model [33], the travel cost [35], the total delivery costs subject to a delivery time limit and the overall delivery time subject to a budget constraint [5]. Adopted techniques include Mixed Integer Linear Programming (MILP) [35, 41–43], Variable Neighborhood Search (VNS) [35], Markov decision [37], Adaptive and Large Neighborhood Search meta-heuristics [41, 42], branch-and-price algorithm for multiple deliveries per trip [39], and Ant Colony Optimization [35].

For example, simultaneous pickup and delivery in the context of multi-visit flexible docking have been addressed in [42], by combining MILP and adaptive large neighborhood search. The routing model proposed in [41] aims to serve multiple consumers by considering synchronized Drone, truck operations, and using multiple drones that fly from a (and return to the same) truck. In a similar work, called k-Multi-visit Drone Routing Problem, Poikonen et al. [40] aim to deliver consumers packages by associating each set of drones to a truck. Focusing on the risk of collisions between delivery drones, the authors in [43] have designed delivery routes based on constraints like time-dependent velocity and directions. Their solution consists in transforming the Drone, truck routing design into MIP sub-problems, which are solved in an iterative process, using meta-heuristics. To optimize the number of truck/-drone deliveries and the cost of delivery tasks, Campbell et al. [34] proposed continuous approximation models that fit the hybrid Drone, truck delivery operations.

2.3 Drone-as-a-Service composition approaches

In this class, the principles of service computing [3] have been applied to treat drones scheduling and routing as a traditional service composition problem. We distinguish failure-sentient DaaS composition [15], uncertainty-aware DaaS composition (i.e. weather uncertainty handling) [11], failure-aware reactive composition [50], swarm-based drone service composition [12–15], in-flight energy-driven composition [13], multi-consumers swarm-based delivery with a minimal cost [12], spatio-temporal predictive composition with ML classifiers [11].

In a series of works [2, 4, 8, 9, 15, 16], authors have abstracted the drone-based delivery process as a composition of skyway segments. For example, Shahzaad et al. [2] combined Dijkstra and 3D RTree indexing for the composition of drone services, based on their spatio-temporal features. Their approach was extended in [4], to manage drone services' failures caused by the changing weather conditions. That is by using a skyline method and an adaptive look-ahead heuristic-based multi-armed bandit approach [4], in which a local recomposition of the failed DaaS is triggered after estimating the failure impact of the delivery tasks. Drones' soft failures were also handled in [15], using a federated learning model that predicts drones' failure time and uptime in a weighted continual approach.

Focusing on in-flight energy optimization, and to minimize the number of intermediate stations (i.e. recharging times), Alkouz et al. [13] proposed the concept of "support drones", which are responsible of recharging other drones in a flight formation. Based on the concept of partnership with charging stations' providers, the authors have also combined pruning and voting techniques, with the goal to filter out drone service providers and select the best ones.

In [8], a non-cooperative game algorithm was proposed to select the optimal DaaS composition, while considering the congestion at intermediate stations and the influence of their selection on other drones. Alkouz et al. [9] tackled the case when delivery tasks are beyond the capacity of a single drone, due to the packages overweight or the high number of delivered items. The authors proposed a cooperative approach for swarm-based DaaS composition that considers various constraints (e.g., battery capacity, recharging time). They have also applied a modified A* algorithm to take into account the type of DaaS compositions (sequential or parallel) and the type of swarms (static or dynamic).

2.4 Approaches analysis

Analyzing the drone-based delivery approaches (see Table 1), the focus spans operational efficiency, cost minimization, and optimized route planning for hybrid Drone, truck and/or multi-drone delivery missions. Most studies emphasize route optimization in collaborative settings between trucks and drones, with a recent orientation towards the servicelization of drone delivery tasks (i.e. Drone-as-a-Service). Recently, dynamic scheduling strategies have been incorporated into delivery systems like in [11, 19, 32, 36, 40, 48]. However, they failed to correctly address the uncertainty of stations congestion, drones demands, and environmental conditions, which makes most approaches less adaptive and responsive to the dynamic UAV network. From the literature, we identified the following drawbacks:

DaaS network modeling. Most approaches do not correctly represent the relationships and semantics between the drone environment's entities (drones, charging stations, pads, delivery services, etc.). In fact, a simple graph topology is the common data structure used to represent the connections between charging stations, while useful and auxiliary information (e.g., drone-station flight/charging data) are often ignored in most approaches. We solve this issue by proposing a structured and multi-relational modeling of such entities using knowledge graphs [51] (see Section 4). That ensured an accurate analysis of drones' and stations' features, thus exploiting their latent information (e.g., proximity, conflicts, resource compatibility) in the selection of high-capacity drones and charging stations.

Processing complexity. Scalability is still a common limitation of most approaches due to the high computational cost, particularly when sky networks hold a large number of drones and delivery hubs (e.g., charging stations). Approaches based on Mixed Integer Linear Programming and evolutionary algorithms (e.g., [29, 40]) require computational trade-offs for feasible solutions (i.e., delivery paths and drones scheduling) in larger UAV networks. Decentralized approaches like federated learning have been explored in recent research. However, they are still in their infancy, as they only treat the general aspects of the delivery scheduling process. Existing approaches, except for [8], ignore the proximity between concurrent drones and charging stations in the skyway network. This latter's exploration is a complex and time-consuming task, especially with the ever changing conditions. Since this influences the tracking accuracy of drones states (e.g., battery level) and stations resources (e.g., recharging pads' availability), we propose a proximity-aware knowledge graph embedding method (see Section 5.1) that serves as a pre-processing step to facilitate the arrangement of similar drones and stations in a vector space, and avoid re-exploring the whole DaaS network.

Focused constraints. Current approaches, either they consider the user-side delivery constraints (e.g., delivery cost and time [2, 5, 9]), or they focus on provide-side constraints (e.g., charging capacities, travel cost, flying regulations [15, 19, 28, 31, 34, 35]). However, combining both types of constraints will optimize at best the delivery process. In our work, we exploit the semantics of both drones' and stations' entities, to measure these latter's similarities in terms of needed and offered resources. This stage of our approach can be seen as a subgraph-aware matching that facilitates measuring the proximity between drones and recharging stations, thus, predicting whether a drone can attend a station (see Fig. 3), and optimizing both the flight path and the delivery cost.

DaaS composition and path search. Current approaches tackle the generation of delivery plans as two isolated sub-problems (*sky path search* and *drones assignment*). That is by searching the shortest skyway path, for which a drone formation is allocated (e.g., uncertainty-aware DaaS scheduling [11]), or by first assigning a set of drones as an input to the routing task (e.g., k-multivisit DRP [40]). However, the former approach often leads to conflicting drone formations with high waiting and recharging times, whereas in the later approach drone formations face the lack of resources on

proximity stations, which often leads to longer skyway paths and, consequently, SLA violations. Our approach solves these issues through an embedding model (see Section 5.1), that maps both low-congestion stations and high-flight capacity drones to the same search space (see Fig. 2). In this way, the selection of drone services and skyway segments will be treated as two closely related sub-problems.

Another major limitation lies in the lack of machine learning support, as most researchers exploit evolutionary techniques and meta-heuristics to optimize delivery routes. Machine learning models are crucial in endowing delivery systems with predictive capabilities like failure prevention, stations congestion, and uncertainty handling, which will ensure efficient DaaS compositions and resilient delivery missions. Finally, a prevalent drawback of current research is the limited consideration and the lack of handling of drone failures and congestion situations in core stations, which is pivotal and critical in case of multi-drone and swarm-based composition of delivery services [4, 15].

Table 1 Summary of drone-based package delivery approaches

Appr.	Goal	Mission type	Service	Parameters/constraints	Path search	Used technique(s)
[24]	Route optimization	Package delivery	Drone, truck	Delivery cost	Greedy Randomized Adaptive Search	Mixed integer programming
[25]	Route optimization	Package delivery	Drone, truck	Battery capacity, drone/truck speed	Heuristic search	Mixed integer programming
[22]	Route optimization	en-route delivery	Drone, truck	Flight range, timing synchronization, operational cost	Modified GRASP	Lin-Kernighan optimization
[27]	Route optimization	Hybrid delivery	Drone, truck service	Flight range, stations availability, delivery time	TSP variant	Mixed integer programming
[30]	Route optimization	Contactless delivery	Drone, truck	Risk and make-span	TSP variant	Robust and multi-objective optimization
[19, 48]	Parallel scheduling	Hybrid delivery	Truck - multi-drone	Energy, flight restrictions and range, drone endurance, delivery time	TSP solver	MILP, Mathematical programming
[44]	Task assignment, route planning	Pickup/ delivery	Drone, truck	Drone capacity, task demand, geographical restrictions	Local search algorithm	Simulated annealing, variable neighborhood descent
[21]	Optimize launch sites and drones per truck	Package delivery	Drone+truck	Delivery time and cost, energy consumption	TSP, GA	k-means, genetic algorithm
[49]	Route optimization	Hybrid delivery	ruck - multi-drone	drones/trucks speeds and capacities, operational cost	Dynamic hybrid TSP algorithm	Fitness-distance balance-based evolutionary search
[28]	Route optimization	Last-mile delivery	Drone+Truck	Energy consumption, flight restrictions and range, payload	Heuristic search	Constructive methods
[20]	Route optimization	Last-mile delivery	Multi-drone, truck	flight range, synchronization of truck and drone movements	VRP variant	MIP
[29, 41]	Optimize/coordinate Drone, truck routing	Multi-consumer delivery	Multi-drone, truck	Arrival time, battery capacity, flight range	Adaptive Insertion	MILP, Tabu search, drone truck route construction, VNS
[5]	Route optimization	Hybrid delivery	multi Drone, truck	Budget constraint, payload capacity, energy consumption, flight range	Clarke and Wright savings algorithm	Capacitated VRP, linear model
[32, 40]	Scheduling and synchronized routing	Multi-visit delivery	Multi-drone, truck	Carbon emissions, delivery cost, regulatory and environmental factors, battery	Route-Transform Shortest Path, MIP	Combinatorial optimization, MIP, VNS
[33]	Route optimization	Last-mile delivery	Drone, truck	customer time windows, maximum drone travel distance	Clarke-Wright savings algorithm.	Combinatorial optimization, VRP variant
[34]	Hybrid delivery optimization	Home and business deliveries	Drone, truck	Marginal stop costs, relative operating costs, spatial density of customers	-	Approximation models, continuous approximation modeling
[35]	Route optimization	Hybrid delivery	Drone, truck	Travel cost, flight range, battery capacity, time windows	VNS	Ant colony optimization, MILP
[36]	Multiple visits scheduling	Multi-modal drop-pickup	multi-Multi-drone, truck, multi-depot	Time window, multiple visits, sequence-dependent setup times	Unrelated parallel machine scheduling	Constraint programming

Table 1 (continued)

Appr.	Goal	Mission type	Service	Parameters/constraints	Path search	Used technique(s)
[37]	Route optimization, vehicle assignment	Same-day delivery	Drone, truck	Time-sensitive delivery, vehicle capacity	Nearest-neighbor search, FCFS	Markov decision, parametric policy function approximation
[38]	Route optimization	Pick-up/ delivery	Drone, truck	Delivery cost, flight range, payload, regulatory rules (line of sight)	Clarke and Wright algorithm	MIP, multi-modal cost-saving search
[39]	Route optimization	Multiple last-mile deliveries	Drone, truck	Delivery time	-	MIP, branch-and-price
[42]	Drone routing	Multi-visit simultaneous pickup/ delivery	Multi-drone, truck	Transportation costs and time, time window	Adaptive and Large Neighborhood Search	MILP
[43]	Collision-free trajectory planning	Package delivery	Drone, truck	time-dependent velocity and directions, risk of collisions	-	MILP, Time-space network, meta-heuristics
[11]	Uncertain scheduling and composition	Package delivery	DaaS	Spatio-temporal information, drone capacity, weather	Uncertainty-aware A*, Greedy Dijkstra	ML classifiers
[50]	Failure-aware reactive composition	Package delivery	DaaS	Skyway segments unavailability	-	Radius-based, cell density-based algorithms
[12]	Swarm-based composition	Multi-consumers delivery	Homogeneous swarm	Congestion, Strict time of delivery, time overlapped requests, inter-dependency of requests, network contentions	Modified A*	Heuristic-based and greedy allocation
[13, 14]	Energy-driven composition	Multi-package	Swarm (support drones)	Energy consumption, recharging times, number of stations, consumers preferences, provider partnership	Modified A*	Priority-based and Fairness-based heuristics, density-based pruning and voting
[15]	Failure-sentient composition	Package delivery	Swarm-based	Failure severity, drones' estimated failure time and up-time, soft failures	-	Weighted continual federated learning
[9]	Sequential/parallel composition	Package delivery	Multi-drone	Battery capacity, recharging time	A* search	Boids algorithm
[8]	Drone service composition	Package delivery	Multi-drone	Stations congestion, charging constraints	-	Non-cooperative game theory
[16]	Route optimization	Package delivery	Single drone	delivery time and distance, congestion	A* search	Line-of-sight heuristic
[2, 4]	Failure-aware local recomposition	Package delivery	DaaS	Spatio-temporal features, delivery cost, QoS, drone failures, failure impact	Dijkstra	3D RTree indexing, brute-force method,Skyline methods, multi-armed bandit

3 Problem formulation

The problem of drone services composition is defined as follows: *Given a set of DaaS with their capacities and features, a set of stations with their charging features, a set of delivery tasks with their cost and time constraints, the goal is to find the optimal skyway path and set of drone services that deliver the consumer's requested packages in a minimum time and cost, while taking into account the drones capacities and the delivery constraints.*

Input. The DaaS environment consists of a set of drones $\mathcal{D} = \{d_1, d_2, ..., d_n\}$, a set of delivery services $\mathcal{S} = \{daas_1, daas_2, ..., daas_m\}$, and a set of recharging stations $\mathcal{RS} = \{rs_1, rs_2, ..., rs_p\}$. Each of these latter hosts a set of recharging pads $\mathcal{P} = \{p_1, p_2, ..., p_q\}$.

Definition 1 (*Delivery request*). A Delivery request $\mathcal{DR}$ is a tuple $< DS, DT, D^F, D^q >$, where DS and DT denote respectively the delivery source and destination, $D^F = \{it_1, it_2..., it_w\}$ represents the delivered packages, with each item it_i $(1 \leq i \leq w)$ is characterized by its *weight* and *quantity*; $D^Q = < T, cost >$ denotes the consumer's delivery time and cost constraints. $T = [\tau_b, \tau_e]$ $(\tau_b \leq \tau_e)$ denotes the preferred delivery time interval.

Definition 2 (*Drone-as-a-Service*). A $\mathcal{D}aaS$ is defined as a tuple $< \mathcal{F}^s, \mathcal{F}^s\mathcal{Q}^s >$, where $\mathcal{F}^s = \{f_1^s, f_2^s, ..., f_z^s\}$ denotes the delivery operations offered by a drone (e.g., first-aid kits). $\mathcal{Q}^s = \{q_1^s, q_2^s, ..., q_z^s\}$ denotes the non-functional properties of the delivery service $\mathcal{D}aaS$ (e.g., delivery cost, payload, max speed, flying range, battery capacity).

Definition 3 (*Recharging station*) is a tuple $rs = \langle F^r, Pad \rangle$, where $F^r = \{f_1^r, f_2^r, ..., f_p^r\}$ denotes the recharging station's features including the max power, the capacity and the type of station (mobile or fixed); a set of charging pads $Pad = \{p_1, p_2, ..., p_q\}$ available at rs. Each pad p_i $(1 \leq i \leq q)$ is characterized by a type (e.g., wireless charging), a lightweight, a max power, etc.

Output. The user's delivery request is satisfied by a set of drone services, called *composite DaaS* (see Eq. 1), and a set of skyway segments representing the delivery path from a source node to a destination (see Eq. 2), as mentioned in the delivery request (see Def. 1).

$$Delivery : D^F \rightarrow DaaS \tag{1}$$

where $it_j \mapsto Delivery(it_j) = daas_i$, $1 \leq i \leq n$, $1 \leq j \leq w$.

$$Path : DaaS \rightarrow Segment \tag{2}$$

where $daas_i \mapsto Path(daas_i) = Seg_k$, $1 \leq i \leq n$, $1 \leq k \leq p$.

Definition 4 (*Composite DaaS*) A Composite drone service $\mathcal{C}DaaS = \{daas_1, daas_2, ..., daas_m\}$ $(1 \leq m \leq n)$ is an aggregation of m delivery missions, achieved in a sequential or parallel way by a drones formation.

Definition 5 (*Skyway segment*) represents a connection between two stations. A skyway segment $\mathcal{S}eg_{ij}$ is a 3-tuple $< rs_i, F, rs_j >$, where $rs_i, rs_j \in \mathcal{RS}$ are recharging stations, and F denotes the skyway segment's features, such as the distance and the wind speed.

Constraints. We distinguish two types of constraints:

- *Consumer constraints:*
  - *Delivery cost:* indicates the maximum cost to deliver one or more packages (D^F) from a delivery source to a delivery target withing a given period of time.

$$\forall\ daas_i \in S \quad \sum_{it_j \in D^F} cost(it_j) \leq cost \mid Delivery(it_j) = daas_i \tag{3}$$

  - *Delivery time:* is the preferred time interval $\mathcal{T}$ to deliver the requested package D^F from a source to a destination.

$$\forall\ d_i \in \mathcal{D}, rs_j \in \mathcal{RS}\ \sum_{rs_j(d_i) \in \mathcal{RS}} time(d_i, rs_j) + \sum_{Seg_k \in Seg} Seg_k \leq \mathcal{T} \mid Path(d_i) = Seg_k \tag{4}$$

- *Provider constraints:*
  - *Recharging time:* is the maximum time (T_{max}) required to recharge a drone $d_i \in \mathcal{D}$ in a recharging station $(rs_j \in \mathcal{RS})$. Minimizing T_{max} will help reduce the total delivery time.

$$\forall\ d_i \in \mathcal{D}, rs_j \in \mathcal{RS} \sum_{rs_j(d_i) \in \mathcal{RS}} time(d_i, rs_j) \leq T_{max} \mid recharging(d_i) = rs_j \tag{5}$$

  - *Payload weight:* is the max weight allowed to deliver a set of items $(it_j \in D^F)$ by a drone $d_i \in \mathcal{D}$ with a maximum payload capacity pl_i.

$$\forall\ d_i \in D \sum_{it_j \in D^F} \leq pl_i \mid Delivery(it_j) = d_i \tag{6}$$

Next, we propose a knowledge graph for drone services (called *DaaSKG*), to model DaaS information and the relationships between sky network entities (drones, locations, recharging stations, delivery services, recharging pads, etc.).

4 Sky network modeling

To take into account the multi-relational nature between drones, delivery services and recharging stations, we represent the skyway network data as a *knowledge graph*.

4.1 Knowledge graphs for drone-based delivery

As a de facto standard to represent and store complex relationships between entities [51], knowledge graphs have shown a great potential in supporting many decision-support and intelligent systems. Proposed for the first time by Google in 2012, then adopted by big companies (e.g., LinkedIn, IBM, Microsoft, Yahoo, Facebook and Wikimedia), this new kind of structured semantic knowledge bases has been exploited in various fields including item recommendation, targeted advertising, business-oriented networks, navigation and location-based services, etc.). Typical knowledge graphs include Wikidata, OpenIE, Freebase, Cyc, NELL, DBPedia, YAGO, MetaWeb, Prospera, Knowledge Vault, GeoNames, ConceptNet, etc.

Compared to traditional graph structures which focus of the connectivity between entities (vertices) with a low encoding of the semantics of their relationships (edges), knowledge graphs have originally emerged as a kind of knowledge bases with multi-relational, hierarchical and semantic capabilities [51]. As a powerful mean to model and analyze complex UAV networks, a knowledge graph does not only represent sky network entities (e.g., drones, stations, drone services), but also multiple, typed, and hierarchical relationships between them (e.g., flight segments, no-fly zones). Also, the evolving and inter-operability nature of knowledge graphs allows integrating new entities and relationships from various sources, like new drone/station features, operations, or flight history. This is not the case of traditional graph structures which often require a predefined schema for data integration. Moreover, modeling sky network information as a knowledge graph is beneficial thanks to its querying support. For example, using languages like SPARQL, we can extract various insights (e.g., low-congestion stations, skyway segments with low wind effects, etc.) and derive useful knowledge for the drone-based delivery missions.

Additionally, a knowledge graph representation will endow the UAV delivery system with inference and reasoning capabilities (e.g., predicting flight paths, or stations overload), which will effectively guide the drone services composition process. Thanks to the semantic and multi-relational modeling of DaaS related entities, applying network representation learning techniques [52], like metapath2vec in our work (see §5.1), will help map the knowledge graph entities in a vector representation, facilitating their processing based on their closeness in the embedding space [52, 53]. Besides the above advantages, knowledge graphs acceptance by leading UAV companies (e.g., Amazon Prime Air, Wing, Zipline) is attributed to its ability to enhance data management (e.g., drones/stations clustering), analytics queries (e.g., stations demands, failure prediction), knowledge discovery (e.g., latent connections, flight patterns, drone formation patterns), decision making (e.g., en-route update of flight paths) and recommendation (e.g., best drone services with low failure rates).

4.2 Overview of the DaaSKG

The proposed *DaaS knowledge graph* (see Def. 6) is a multi-relational directed graph with typed edges between sky network entities. It covers various types of entities and information about the DaaS environment, such as drones features and flight capacities, delivery operations and quality levels, recharging pads' resources, relations between entities, skyway segments, etc. For example, the edges between drones and delivery operations indicate that the drone is providing these services, whereas the edges between recharging stations and pads indicate the type of resources available at each station. By annotating the relations between those entities with auxiliary information, we obtain a rich and heterogeneous information network, as depicted in Fig. 1.

Definition 6 (*DaaS Knowledge Graph*) a *DaaSKG* is a multi-relational directed graph $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{D}^+)$, where $\mathcal{E} = <\mathcal{E}_d, \mathcal{E}_s, \mathcal{E}_r, \mathcal{E}_p, \mathcal{E}_f>$ is a set of entities denoting the drones ($\mathcal{E}_d$), delivery services ($\mathcal{E}_s$), recharging stations ($\mathcal{E}_r$), recharging pads $\mathcal{E}_p$, and features ($\mathcal{E}_f$); $\mathcal{R} \subseteq \mathcal{E} \times \mathcal{E}$ is a set of typed relations between entities; and $\mathcal{D}^+$ is a set of triples (facts) expressing the entities interconnections.

Definition 7 (*Fact*) a fact in the *DaaSKG* is a tuple $f = (e_i, r, e_j)$, where $e_i, e_j \in \mathcal{E}$ are drones, recharging stations, delivery services or features; $r \in \mathcal{R}$ is a typed relation between e_i and e_j. The set of facts in the *DaaSKG* is denoted by $\mathcal{D}^+ = \{(e_i, r, e_j)\}$.

Definition 8 (*Relation*) A relation $r \in \mathcal{R}$ in the *DaaSKG* is a link between two entities e_i and e_j. It is defined as $r : e_i \xrightarrow{r} e_j$, where $e_i, e_j \in \mathcal{E}$ ($\mathcal{E} = <\mathcal{E}_d \cup \mathcal{E}_s \cup \mathcal{E}_r \cup \mathcal{E}_f>$). Each relation between two entities is augmented with auxiliary information. Depending on the entities types, the relation function $f(r)$ has the following values.

$$f(e_i, r, e_j) = \begin{cases} ATTEND & \text{if } e_i \in \mathcal{E}_d \wedge e_j \in \mathcal{E}_r \\ PROVIDE & \text{if } e_i \in \mathcal{E}_d \wedge e_j \in \mathcal{E}_s \\ SkySegment & \text{if } e_i \in \mathcal{E}_r \wedge e_j \in \mathcal{E}_r \\ D-SIMILAR & \text{if } e_i, e_j \in \mathcal{E}_d \\ S-SIMILAR & \text{if } e_i, e_j \in \mathcal{E}_s \\ R-SIMILAR & \text{if } e_i, e_j \in \mathcal{E}_r \\ F-SIMILAR & \text{if } e_i, e_j \in \mathcal{E}_f \\ BELONG & \text{if } e_i \in \mathcal{E}_p \wedge e_i \in \mathcal{E}_r \end{cases}$$

Fig. 1 shows that several entities are involved in the construction of the DaaSKG. These include the drones and

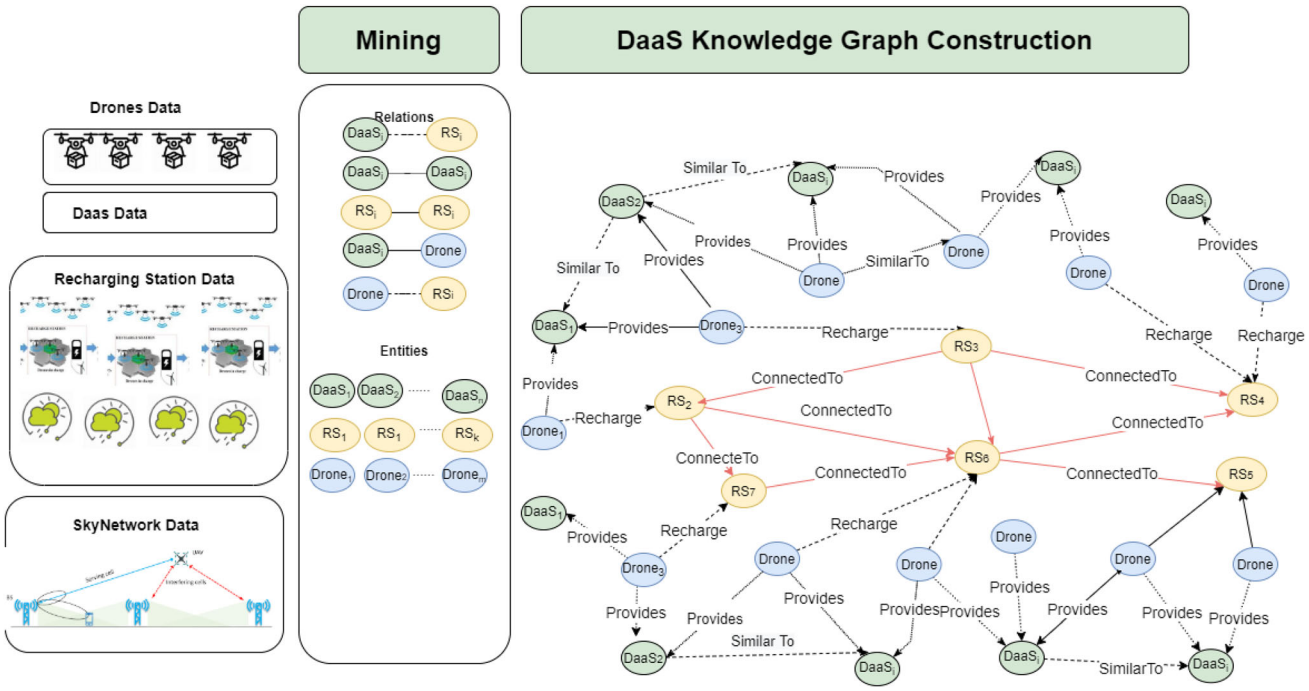

Fig. 1 Example of DaaS knowledge graph

their delivery services, the recharging stations and their pads, the functional and nonfunctional descriptions (features) of those elements. We also distinguish different types of relations including *Drone-Drone*, *Drone-DaaS*, *Station-Pad*, *Station-Station*, etc. For example, a fact $< d_i, r, s_j >$ indicates that a delivery service $s_j \in \mathcal{E}_s$ is provided by a drone $d_i \in \mathcal{E}_d$. In this case, there will be a typed edge in the *DaaSKG* relating the nodes d_i and s_j. This edge expresses a service delivery relation through the positive fact $\tau^+ = (d_i, PROVIDES, s_j)$. Also, the relation between a delivery service s_j and a feature f_k^s (e.g., delivery cost) is denoted by the triple $(f_k^s, DESCRIBE, s_j)$, to indicate that the service s_j is characterized by the feature f_k. Table 1 gives some examples of relations between entities within the DaaSKG. In these examples, the facts are associated with auxiliary information in the form (e_i, r, e_j, aux), where e_i and e_j denote, respectively, the subject (head) and object (tail) entities linked via the relation r, whereas aux denotes the set of auxiliary information about that relation.

4.3 DaaSKG construction

To populate the DaaSKG, we mined several data sources that contain relevant information about drones (e.g., features, capacities, and delivery operations), recharging stations (e.g., locations, states, pads), and skyway segments (e.g., distance, flight restrictions). Such information is categorized into entities, typed relations, and auxiliary information (see Table 1). Then they are combined to form the facts in the DaaSKG. This process is referred to as knowledge graph construction and consists of two main operations: knowledge population (by extracting data from various resources) and knowledge completion (by analyzing the extracted data and inferring latent knowledge information) [54].

The DaaSKG construction involves the following steps to extract sky network entities and define meaningful relationships between them:

1) *Sky network data collection:* this step aims at gathering drone features and related data from various sources. These include flight logs, missions history (flight paths, failure rates, packages information), etc. Stations-related information include their features (e.g., geo-location, charging typed), charging pads and capacities (e.g., average waiting/charging times), and traffic history. Other essential information include sky segments, flight regulations and restrictions (i.e., no-flight zones), in addition to the environmental conditions (e.g., weather and wind effects). The collected data origin from UAV delivery systems, drone/station service provides, in addition to environmental IoT sensors. Before their mining, the collected data undergo a pre-processing step consisting of cleaning operations (e.g., removing duplicate data), transformations (e.g., latitude/longitude coordinates of all stations), and normalization (e.g., distance in km, energy in kWh).

Table 2 Example of facts in the DaaSKG

Subject	Relation	Object	Auxiliary information
d_1	ATTEND	rs_1	[Battery:5000mA, Payload:15kg, Time:2.5H]
d_1	PROVIDE	$daas_1$	[Price: 13.5, Payload : 3.5pounds]
$daas_1$	S-SIMILAR	$daas_2$	[Payload:6.6 pounds, Flight: 16.2 miles]
SkyPad	BELONGTO	SkyCharge	[Power: 10kv]
rs_1	SkySegment	rs_2	[Distance: 10 miles]

2) *Entity/Relation extraction:* at this stage, the core entities and typed relations that will draw the main topology of the DaaSKG are identified from the pruned data. These mainly include drones, drone services, stations, and charging pads. To ensure an accurate and meaningful definition of the relations between those entities, machine learning techniques and data mining techniques can be applied, allowing to identify the typed relationships from data patterns (e.g., station-station, drone-service, station-pad, etc.). For example, the *drone-service* patterns reflect the typed relation *PROVIDE*, for which a drone offers a delivery service with particular QoS information (e.g., cost, rating, etc.); whereas the *station-station* patterns indicate the sky segments (i.e. flight paths) between two stations. As for the *drone-station* patterns, they may indicate particular charging features for a set of drones with specific charging needs (e.g., specific voltage). Therefore, such patterns are expressed by the typed relation *ATTEND*.
3) *Entity enrichment:* this step aims at enriching each entity of the DaaSKG with relevant features and auxiliary information. For example, the entities representing drones and their services can be augmented with measurable (e.g., failure rate, battery capacity) and non measurable (e.g., cost, average speed) properties. While average waiting/charging times, charging type, and voltage features can characterize stations an their charging pads. Other auxiliary information can enrich the typed relations, such as the properties of the flight segments composing the sky network (e.g., no-fly zones, weather conditions/forecasts, distance, etc.).
4) *Triples formation:* this final step generates the basic units of knowledge (i.e. facts) in the DaaSKG, which are structured as e_i,$r^(t)$, e_j. The initial triples are based on the extracted entities and typed relations, and can be exploited to infer latent connections and complete the DaaSKG structure.

To ensure an efficient definition of delivery plans, the DaaSKG processing must not be limited to the data initially mined from the skyway network. Rather, additional knowledge (e.g., proximity between drones and stations) must be inferred from the extracted information. Besides the generated triples, we augmented the DaaSKG by inferring latent relations between its entities, based on existing facts and similarities between their node characteristics. To achieve this goal, we adopt knowledge graph embedding (KGE) [17] as an effective and scalable means for predicting the hidden connections and enriching the DaaSKG structure (see §5.1).

5 DaaS composition process

As described in §3, fulfilling a consumer's delivery request involves identifying a set of high-quality DaaS along with an optimal delivery path through the skyway network. To achieve this goal, user and provider constraints are addressed through a three-step process for drone services composition:

1) *Extraction of candidate drones and stations:* In this step, the delivery space is refined and reduced to keep only the drones and recharging stations that will participate in the delivery task, i.e. the nearest stations to the delivery source and target. That is by measuring the similarity between drone services and the proximity between recharging stations. We followed a metapath-guided learning to translate the DaaSKG into a low-dimensional vector space, which facilitates the identification of close candidate drones and stations.
2) *Proximity-aware DaaS Composition:* Based on the extracted subsets of drones and recharging stations, as well as their proximity degrees, a proximity-aware algorithm is defined to combine the best available drone services that can achieve the delivery mission in a minimum delivery time and cost.
3) *Skyway path selection:* Since the composite DaaS may reach the delivery target through different skyway paths, this step aims at selecting the skyway segments that minimize, not only the distance between the delivery source and target, but also the

recharging frequency and, consequently, the waiting time at the intermediate recharging stations.

5.1 Extraction of candidate drones and stations

In this section, we frame the DaaS selection as a proximity-aware matching problem to facilitate measuring similarity between DaaSKG entities. This approach helps identify nearby drones and recharging stations that will participate in the delivery operations.

A first step, towards finding a high-quality DaaS combination, is to extract the appropriate drones (in terms of flight capacity and QoS levels) and the closest recharging stations. To achieve this goal, we adopt a knowledge graph embedding (KGE) model that allows a guided exploration of the DaaSKG entities. KGE is an effective mean, not only to capture and encode structural and contextual information (unlike processing raw node features), but also to infer latent features and to learn rich representations and semantics of graph-like information networks [17]. In a typical network representation learning (NRL) process [52], sampling methods (e.g., edge sampling, biased sampling) are first applied to explore the knowledge graph's local structure (e.g., node degrees, node neighbors) or global structure (e.g., multi-hop neighborhood, node ranks), while considering different proximity levels (e.g., first-order, higher-order, or proximity degree). Then, embedding models (e.g., graph neural networks, probabilistic models) and optimization techniques (e.g., hierarchical softmax, attention mechanism) are applied to infer hidden feature vectors and obtain node representations. Translational distance models, semantic matching models, and recently random walk- and deep learning- based techniques are popular examples of KGE models [17, 52]. These latter have successfully been applied in various fields, including node classification, link prediction, ontology population, community detection, recommender systems, and relational learning [52].

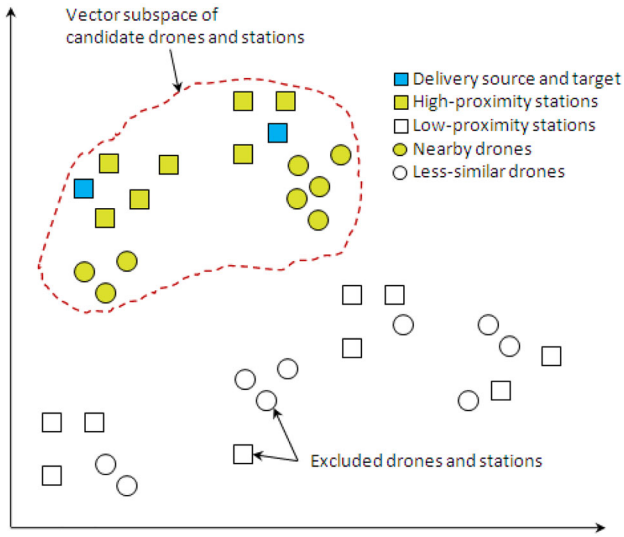

Fig. 2 Low-dimensional embedding space of the DaaSKG

5.1.1 DaaSKG vector subspace

In out work, the KGE model allows mapping the DaaSKG entities (e.g., drones, delivery operations, recharging stations and pads) as close as possible in a low-dimensional semantic embedding vector space, according to the similarity between their features. Fig. 2 depicts the projection of DaaSKG entities into the vector space. The vector representations of the best candidate drones and recharging stations (i.e. with the highest proximity degrees) are marked as olive ◯ and □, respectively. These vectors form an *embedding subspace* that corresponds to the best skyway sub-network to satisfy a delivery request. We also notice, from Fig. 2, that some drones and stations (white ◯ and □) have low proximity degrees. That is understandable because they do not provide the requested delivery services or they do not satisfy the recharging constraints.

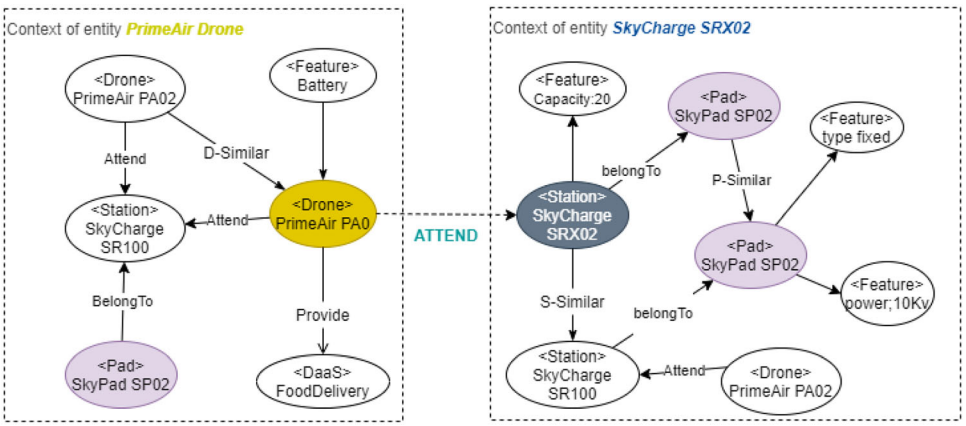

Fig. 3 Example of latent relation between a drone and a station

Table 2 Concepts mapping between metapath2vec and DaaS composition

KGE model	DaaS composition problem
Knowledge graph	DaaSKG: Skyway information network, including stations, drones, services, and features (see Def. 6)
Nodes	Representative entities of charging stations and pads, drone services, etc. (see Fig. 1)
Edges	Skyway segments and typed relations between DaaSKG entities (see Def. 8)
Meta-path	Abstract set of typed entities and relations, i.e a predefined pattern to which adhere a sequence of nodes (see Fig. 4)
Transition probability	Probability of obtaining the next node to a given one given a metapath (see Eq. 7)
Embedding space	Vector representation of the DaaSKG entities (see Fig. 2)
Vector subspace	Subset of high-proximity drones and stations (see Fig. 2)
Neighborhood	A node's context at a specific depth (see Fig. 3)

Consequently, they are excluded from the drone services' composition process.

To achieve the embedding task, we adopt a NRL method called metapath2vec [53], which we use to perform guided random walks and compute the proximity degrees between the DaaSKG entities (see Table 2). This step is essential to extract the optimal vector subspace, which contains the drone services with the highest flight and delivery capacities, as well as the stations that satisfy those drones' recharging requirements. The proximity evaluation between two nodes in the DaaSKG (e.g., two drones or two recharging stations) can be treated as a subgraph matching problem. In this case, higher-order methods for proximity measure could be a good way to learn the vector representations of DaaSKG entities. These latter's embeddings will be encoded to have similar vectorized forms and high proximity degrees, if they share similar features, which facilitates their filtering and use in the DaaS composition process.

For example, to predict whether a drone d can visit a charging station rs, a proximity evaluation must be performed based on the context of these two entities. Fig. 3 depicts the neighborhood of the drone entity *PrimeAir* and the recharging station *SkyCharge SRX02*. The relation *D-SIMILAR* indicates that the drone *PrimeAir PA01* shares similar features with the drone *PrimeAir PA02*. This latter has previously attended the station *SkyCharge SR100*, which offers the same charging capacities as in *SkyCharge SRX02* (e.g., pads, recharging type, voltage). These common neighbor entities and features between the two nodes' contexts reflect the high similarity between the drone's recharging requirements and the station's charging capacities, which translates into a high proximity degree between *PrimeAir PA01* and *SkyCharge SRX02*. Consequently, these latter's vector representations will be close in the embedding space, and the latent relation (*ATTEND*) between them will be inferred. In the same way, two drones' vectors are close in the embedding space (i.e. have a high proximity degree), not only if they have common features and delivery services, but also if they have previously attended similar recharging stations. That reflects their similar recharging needs (e.g., required voltage and type of charging pads). Likewise, the vectors of stations having similar charging capacities (i.e., charging pads' types, power, voltage, etc.) tend to be close in the embedding space.

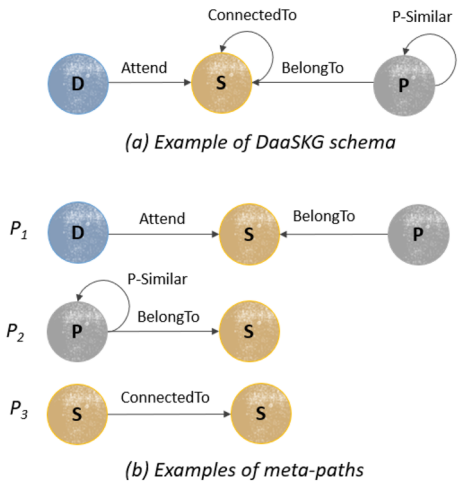

Fig. 4 Example of DaaSKG meta-paths

5.1.2 Meta-path based random walks

To encode drones' and stations' embeddings, we apply guided random walks (i.e. node sequences) on the DaaSKG nodes. Each extracted sequence must adhere to a predefined pattern (also called metapath) of typed nodes and edges. Like depicted in Fig. 4, meta-paths define meaningful node sequences in the DaaSKG. For example, $\mathcal{P}_1$: *(Drone-Station-Pad)* represents all the node sequences where a drone D accesses a specific pad P within a charging station S. While the metapath $\mathcal{P}_2$: *(Station-Pad-Pad)* indicates that a drone can move from one charging pad to another within the same station, in order to fully charge its battery an resume its delivery mission. As for the

node sequences of the metapath $\mathcal{P}_3$: *(Station-Station)*, they express the skyway segments that may be traversed by drones.

In our view, nodes that frequently adhere to metapath-guided random walks usually have similar contexts, and their vector representations will be close in the embedding space. For example, the frequent co-occurrence of a charging station S in more than one node sequence reflects its high charging capacity and ability to serve the flying drones. Such structural and semantic relationships between drone services and charging stations translates into a proximity degree, which denotes the likelihood of involving a drone in the delivery mission, and the probability of selecting an intermediate charging station for the final delivery path.

Given a meta-path scheme $\mathcal{P}: V_1 \xrightarrow{R_1} V_2 \xrightarrow{R_2} ...V_t \xrightarrow{R_t} V_{t+1}... \xrightarrow{R_{l-1}} V_l$ defined by domain experts, the DaaSKG exploration is guided by a transition probability $p(v^{i+1}|v_t^i, \mathcal{P})$ (see Eq. 7 [53]), denoting the likelihood of reaching the next entity v^{i+1} (e.g., station) to a given node v_t^i (e.g., drone).

$$p(v^{i+1}|v_t^i, \mathcal{P}) = \begin{cases} \frac{1}{|N_{t+1}(v_t^i)|} & (v^{i+1}, v_t^i) \in \mathcal{R}, \quad \emptyset\ (v^{i+1}) = t+1 \\ 0 & (v^{i+1}, v_t^i) \in \mathcal{R}, \quad \emptyset\ (v^{i+1}) \neq t+1 \\ 0 & (v^{i+1}, v_t^i) \notin \mathcal{R} \end{cases} \tag{7}$$

Here, t is a node type at time i, and (v^{i+1}, v_t^i) is a transition that eventually belongs to the relations set $\mathcal{R}$. $p(.) = 0$, *if* $(v^{i+1}, v_t^i) \notin \mathcal{R}$, or $(v^{i+1}, v_t^i) \in \mathcal{R} \ \wedge \ t \neq$ expected node v^{i+1}, given $\mathcal{P}$ (e.g., $(rs_2, rs_3) \notin \mathcal{R}$ in Fig. 5).

5.1.3 Skip-gram-based heterogeneous representation

To correctly model the context of each DaaSKG entity, we used Skip-Gram. This learning method, based on a window size, helps determine the depth of neighborhood extraction for each DaaSKG entity v of type t (e.g., drone service), which maximized the closeness between this latter and its neighbors (e.g., similar DaaS, drone features, previously targeted stations). Seen the DaaSKG heterogeneity, Eq. (8) [53] is used to maximize the probability of reaching a neighbor node $c_t \in N_t(v)$

$$\arg\max_{\theta} \sum_{v \in \mathcal{E}} \sum_{t \in T_v} \sum_{c_t \in N_t(v)} log\, p\,(c_t|v;\theta) \tag{8}$$

Here, $p\,(c_t|v;\theta)$ is calculated through a *softmax* function (see Eq. 9), by transforming the input vectors (i.e. v's neighborhood) to values in [0,1].

$$p(c_t \mid v, \theta) = \frac{\exp\ X_{c_t} . X_v}{\sum_{u \in V} \exp X_{c_t} . X_v} \tag{9}$$

Given an input node v (e.g., *PrimeAir PA0* in Fig. 3), its context nodes (e.g., *Battery, FoodDelivery, SkyCharge SR100, SkyPad SP02*) are predicted by applying the *arg max* function. That is ensured by a metapath-guided generation of DaaSKG node sequences, which are then fed to

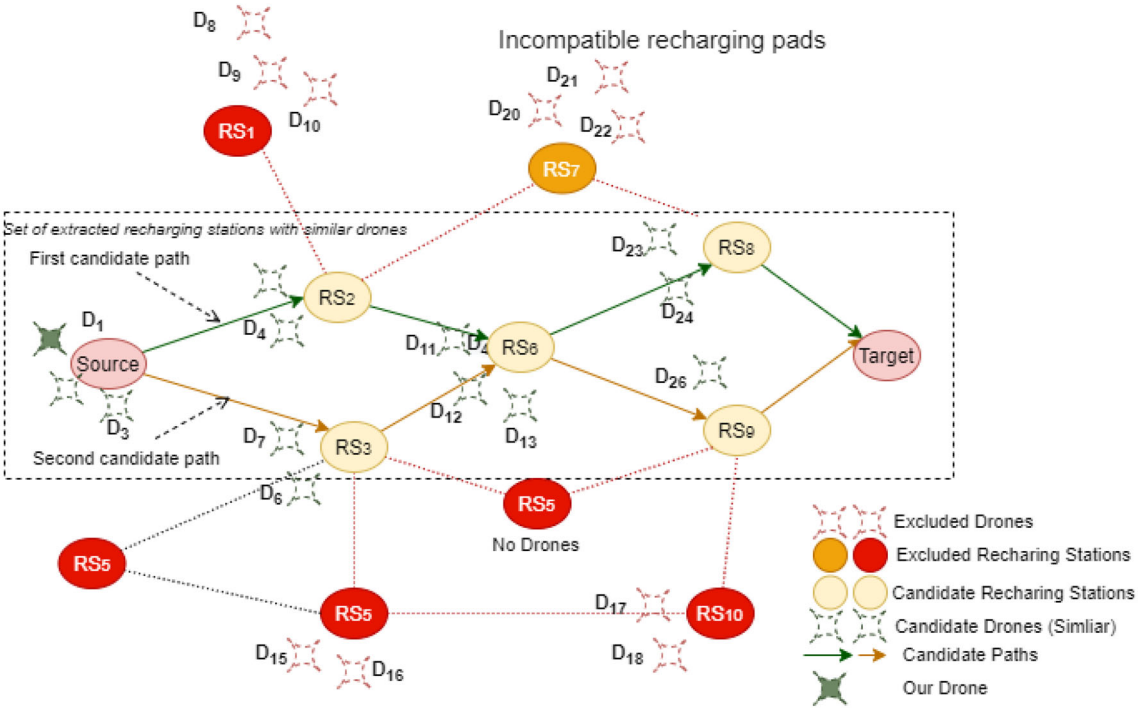

Fig. 5 Candidate drones and stations for the delivery mission

Skip-gram algorithm. This latter's training phase consists of an iterative feed-forward and back-propagation, in which the weights are altered following a calculated error value. Besides the input layer, the KGE model has one hidden layer on which we compute the corresponding weighted sums. The hidden layer dimension depends on the size of the neighborhood extracted for each DaaSKG entity. Add to that, one output layer is associated to each element (weighted sum) of the hidden layer. The output layer also represents the probability distribution of the input DaaSKG nodes., while the output values denote the likelihood that a DaaSKG entity (e.g., *SkyPad SP02*) appears in the first context window.

Once the DaaSKG embeddings are learned, the proximity between node representations is measured along the metapath-guided random walks. Taking the example of the charging station *SkyCharge SRX02* (see Fig. 3), this node's embedding would represent the type of pads, the power capacity, the location, the usage patterns, as its characterizing features (i.e. vector entries). Therefore, the proximity between two stations' vectorized forms is high (close to 1), if their features are similar. Conversely, two drones' vectors that are farther apart in the embedding space will have different entries (e.g., payload, speed, battery capacity, cost).

We use cosine similarity to compute the distance (proximity degree) between DaaSKG entities (see Eq. 10):

$$\text{cosine}(i,j) = \frac{\mathbf{v}_i \cdot \mathbf{v}_j}{\|\mathbf{v}_i\|\|\mathbf{v}_j\|} \tag{10}$$

where $\mathbf{v}_i$ and $\mathbf{v}_i$ are the vector embeddings of i^{th} and j^{th} DaaSKG entities, $\mathbf{v}_i \cdot \mathbf{v}_j$ is the dot product of these embeddings, whereas $\|\mathbf{v}_i\|$ and $\|\mathbf{v}_j\|$ are the magnitudes of $\mathbf{v}_i$ and $\mathbf{v}_j$, respectively. A proximity degree close to 1 indicates similar features and relationships between entities in the DaaSKG, such as two nearby stations or two drones with similar capabilities (e.g., flight range, charging needs, payload).

Next, we show how the embedding space is explored to select the composition of drone services, together with the traversed skyway segments.

5.2 Proximity-aware DaaS composition

This step takes as input the previously generated embeddings of DaaSKG entities, as well as their proximity degrees. Since there exist several skyway paths between two delivery locations, we propose to limit the composition of drone services to the high-proximity entities in the embedding space. The quality of a DaaS composition (a delivery mission) depends on service features. Some of these latter must be minimized (e.g., cost), while others must be maximized (e.g., flight range, battery capacity, max speed). For example, the flight range and the battery capacity should be maximized to reduce the number of intermediate stations, thus, the recharging and total delivery times.

Table 3 Examples of candidate DaaS compositions for a user's delivery request

Delivery plan	Score
$\{\{d_1,\{daas_{11}\}\},\{d_2,\{daas_{21}\}\},\{d_3,\{daas_{31},daas_{32}\}\}\}$	0.751
$\{\{d_1,\{daas_{11}\}\},\{d_5,\{daas_{52},daas_{53}\}\},\{d_6,\{daas_{61}\}\}\}$	0.782
$\{\{d_5,\{daas_{51},daas_{52}\}\},\{d_2,\{daas_{21}\}\},\{d_8,\{daas_{81}\}\}\}$	0.68
$\{\{d_4,\{daas_{42},daas_{43}\}\},\{d_3,\{daas_{31}\}\}\}$	0.709
$\{\{d_9,\{daas_{91},daas_{92}\}\}$	0.88

The likelihood of a drone to participate in a DaaS composition is calculated using Eq. (11):

$$S_{CDaaS} = \frac{1}{n} \times \sum_{i=1}^{n} (w_k \times Q_k^i) \tag{11}$$

where S_{CDaaS} is the composite drone service's score, n is the number of participating drones; $w_k (0 \le w_k \le 1$, $1 \le k \le |\mathcal{F}^d|)$ and Q_k^i denote, respectively, the k^{th} weight (user preference degree) and quality values for the i^{th} drone's k^{th} feature. In this work, we consider four drone features: the total delivery cost, which is equal to $\sum_{i=1}^{n} Co_i$, the minimal flight range $\min_{1...n}(Rn_i)$, the drones' minimal speed $\min_{1...n}(Sp_i)$, and the drones' minimal battery capacity $\min_{1...n}(Bt_i)$. Based on that, Eq. (11) can be translated into:

$$\begin{aligned} S_{CDaaS} = \frac{1}{n} \times (w_1 \sum_{i=1}^{n} Co_i + w_2 \min_{1 \le i \le n} Sp_i + \\ w_3 \min_{1 \le i \le n} Rn_i + w_4 \min_{1 \le i \le n} Bt_i) \end{aligned} \tag{12}$$

We proposed a proximity-aware algorithm that takes as input a *delivery request* and the subset of drone services that belong to the *vector subspace*. The output of Algorithm 1 is a composite DaaS, that will be exploited to determine the delivery path. The first step, aims at generating the candidate DaaS combinations, and computing their scores. Since there is a trade-off between reducing the number of drones involved in the delivery task and satisfying the delivery constraints, we consider the drones' features (payload capacity, delivery cost, battery capacity, and speed) and the stations' available resources (e.g., recharging capacity and recharging time) as two essential criteria in the first filtering step. The number of drones is used to penalize the score of candidate DaaS compositions. Indeed, a high number of participating drones increases the delivery cost and complicates the recharging process (more intermediate stations and additional waiting/charging

time), which explains the reduced score of such DaaS combinations (see Table 3).

Algorithm 1 Composition of drone services for delivery

Require: $\mathcal{D}_{prox}$: candidate DaaSs, $\mathcal{DR}$: delivery request
Ensure: $CDaaS_{best}$: composite DaaS
1: $\mathcal{D}_{prox} \leftarrow$ Sort drones using Eq. (12)
2: $CDaaS \leftarrow \varnothing$ ▷ Set of candidate DaaS compositions
3: $D^F_{temp} \leftarrow D^F$
4: $CD \leftarrow \varnothing$
5: **while** ($D^F_{temp} \neq \varnothing$ AND $Payload(\mathcal{D}_{prox}) >= Weight(D^F_{temp})$ **do** ▷▷ Check if the remaining drones in $\mathcal{D}_{prox}$ can hold the consumer's requested package.
6: **if** d_i can hold it_j **then**
7: Put it_j in d_i ▷▷ Place current item in drone d_i.
8: Remove item it_j from D^F_{temp}.
9: **else**
10: Add d_i to CD ▷▷ Add full drone to the CDaaS.
11: Remove d_i from $\mathcal{D}_{prox}$ ▷▷ Update the set of available drones.
12: $i \leftarrow i + 1$
13: **end if**
14: **end while**
15: **if** ($D^F_{temp} = \varnothing$) **then**
16: add CD to $CDaaS$
17: Compute score of CD using Eq. (11)
18: GoTo 5 ▷▷ Generate the next composite DaaS.
19: **end if**
20: Sort DaaS compositions in $CDaaS$
21: **Return** $CDaaS_{best}$

Guided by the above constraints and goals, Algorithm 1 uses Eq. (12) to sort the candidate drones (line 1) that resulted from the DaaSKG embedding step. For each drone (d_i), its offered delivery services ($Daas_i$) are checked against the delivery request (lines 5–6), then eventually added to the current DaaS combination (lines 6–11). Each drone d_i, which holds a subset of the delivered packages $it_j \in D^F$ (line 7), is then saved in the currently generated composite DaaS (*CD*) (line 10). At this stage, a set of candidate CDaaSs is generated by taking into consideration the drones' payload capacity (line 6) and the delivered items' weight (lines 5). The algorithm checks if each DaaS combination (*CD*), once formed, fulfils the user requirements, and a scoring function that takes into account the total delivery cost and the number of drones is called for each composite DaaS (line 17). This routine is repeated as long as the remaining drone services can achieve together the delivery mission (line 18). Finally, the composite DaaS that minimizes the number of drones, as well as the delivery cost is outputted as the best drone formation (line 21).

5.3 Skyway path selection

To find the shortest delivery path w.r.t. the delivery and flight constraints, we proposed an algorithm that takes as input the previously composed *CDaaS*, in addition to the high-proximity stations ($\mathcal{RS}_{prox}$) that resulted from the embedding step (see Section 5.1). Algorithm 2 sorts the recharging stations in a decreasing order of their proximity degrees (line 1). Then, it runs through the subspace of candidate stations, to locate the delivery source *DS* and target *DT*, and to find the possible skyway segments from *DS* to *DT* (see Function *delivery_paths*). Algorithm 2 must also take into consideration the range capacity of each drone in the generated CDaaS (a single drone or a drones formation). To do so, the minimum flight range is compared (line 10) using the distance between the current and next stations (rs_i, rs_j). After that, the algorithm checks if rs_j serves some drones ($d_i \in CDaaS$), that are ready to substitute a low-capacity drone and continue the delivery task (line 11). In this case, the minimum flight range of the drones in *CDaaS* is updated, and the skyway segments' selection routine is repeated until forming the candidate skyway path.

Algorithm 2 Skyway path selection

Require: $CDaaS_{best}$, $\mathcal{RS}_{prox}$, Delivery request $\mathcal{DR}$.
Ensure: Delivery path $path_{opt}$ for *CDaaS*.
1: Decreasing sort of $\mathcal{RS}_{prox}$
2: $Seg^{SK} \leftarrow delivery\_paths(\mathcal{RS}_{prox}, DS, DT)$
3: Sort Seg^{SK} in increasing order of their distance
4: $D_a \leftarrow \emptyset$
5: $S_{max} = 0$
6: **for each** $path \in Seg^{SK}$ **do**
7: Initialize C, T, t_f, t_r
8: **for each** $seg(rs_i, rs_j) \in path$ **do**
9: **for each** drone $d \in rs_i$ **do**
10: **if** ($d.range > seg.distance$ **then**
11: **if** ($D^F - d.payload$) $\neq \varnothing$ **then**
12: $C \leftarrow C + cost(d, rs_j)$
13: $t_f \leftarrow t_f + time(d, rs_j)$
14: $t_r \leftarrow rechargeTime(d, rs_j)$
15: $T \leftarrow T + t_f + t_r$
16: add d to D_a
17: Break when ($D^F = \emptyset$)
18: **end if**
19: **end if**
20: **end for**
21: **end for**
22: $Score \leftarrow 1/(w_1 T + w_2 C + w_3 |D_a|)$
23: **if** ($Score > S_{max}$) **then**
24: $S_{max} \leftarrow Score$
25: $path_{opt} \leftarrow path$
26: **end if**
27: **end for**
28: **Return** $path_{opt}$

The second part of Algorithm 2 consists in the weighted evaluation of each generated path (line 22), based on several criteria (cost, waiting/charging time, numbers of stations, delivery time). The total delivery time mainly depends on the flying time (t_f), the waiting time (t_w), and the recharging time (t_r). Minimizing those values leads to a faster delivery mission. Finally, the skyway path having the shortest distance and delivery time is returned with the selected set of drone services (line 28).

Function Delivery paths()

Require: Sky network $\mathcal{RS}$, source (rs_s), target (rs_t).
Ensure: Delivery path $path_{st}$ for CDaaS.
1: $dist[rs_s] \leftarrow 0$
2: Initialize $dist, \mathcal{RS}_v, \mathcal{RS}_p, Q$ to $\emptyset$ ▷ Distance matrix and the sets of visited, previous, and queued stations
3: $Q \leftarrow rs_s$
4: **while** $Q \neq \emptyset$ **do**
5: $rs_c \leftarrow Q.dequeueStation()$
6: Exit when $(rs_c = rs_t)$
7: **for** each $rs \in \mathcal{N}(rs_c)$ **do**
8: **if** $rs \notin \mathcal{RS}_v$ **then**
9: $d \leftarrow dist[rs_c] + weight(rs_c, rs)$
10: **if** $rs \notin dist$ OR $d < dist[rs]$ **then**
11: $dist[rs] \leftarrow d$
12: $\mathcal{RS}_p[rs] = rs_c$
13: Q.enqueueStation(rs)
14: **end if**
15: **end if**
16: **end for**
17: add rs to $\mathcal{RS}_v$
18: **end while**
19: $path \leftarrow \emptyset$
20: $rs_c \leftarrow rs_t$
21: **while** $rs_c \in \mathcal{RS}_p$ **do**
22: $path.prepend(rs_c)$
23: $rs_c \leftarrow \mathcal{RS}_p[rs_c]$
24: **end while**
25: $path$.prepend(rs_s)
26: return $path$

5.4 Dynamic configuration of DaaS compositions

Because of the highly dynamic nature of the sky network, several changes occur frequently, which may affect the states of DaaSKG entities and, consequently, the initially computed embeddings. Captured changes include the unavailability of charging pads, drone failures, station congestion, sky segments' unavailability or disturbance factors, etc.

Based on the change rate Δ captured in each time window t, we define the new DaaSKG structure $\mathcal{G}'$ as: $\mathcal{G}' = \mathcal{G} + \Delta(t)\mathcal{G}$, where Δ could be a positive change (e.g., newly added station) or a negative change (e.g., removed sky segment). For example, $\Delta = \{\{R : -(SP02, SR100)\}, \{E : -(SRX02, type : wireless)\}\}$ indicates two captured changes, in which the charging pad $SP02$ is no longer available at the station $SR100$ (removed relation), while the pad type became *wireless* for the node $SRX02$. Δ depends on the changes in DaaSKG entities and connections, and it is defined as:

$$\Delta(t) = \frac{|E'(t)| + |R'(t)|}{|\mathcal{E}| + |\mathcal{R}|} \tag{13}$$

where E' and R' denote, respectively, the DaaSKG relations and nodes affected by changes at time frame t. $E' \cap \mathcal{E} = \emptyset, R' \cap \mathcal{R} = \emptyset$ in case of new added relations ($r \in R'$) or nodes ($e \in E'$).

To offer resilient DaaS compositions and efficient delivery plans, the selection of drone services and traversed stations should be driven by the updated embeddings. Since recomputing the embeddings of all DaaSKG entities is a costly operation, we resort to incremental embedding as an efficient solution to handle only the affected entities and relations within the DaaS knowledge graph. Our approach's incremental variant, namely DaaSKG++, implements metapath2vec++ which is the incremental model of metapath2vec [53].

For each node $v_i \in E'$, DaaSKG++ proceeds by updating the transition probability using Eq. 14 to ensure the random walks guided by a specific meta-path m, and involving the entity v_i affected by the change.

$$p(v_{i+1}|v_i, m) = \frac{1}{\mathcal{N}(v_i, m)} \tag{14}$$

Here, $\mathcal{N}(v_i, m)$ denotes the neighbor DaaSKG entities reachable from the node v_i, following the metapath m.

To capture the local and global structures of node sequences after the change $\Delta(t)$, the current embeddings are incrementally updated by first generating new guided random walks for the affected DaaSKG entities/relations. The generated random walks (with length l) must adhere to the original meta-paths, as explained in §5.1.2.

As in the initial embedding model, we used Skip-Gram to update the DaaSKG embeddings of the affected nodes (i.e. drone services and stations). That is by maximizing the likelihood of predicting the context entities $\mathcal{N}(v_i)$ for each $v_i \in E'$, given the optimized parameters θ of the embedding space, and using the softmax function defined in §5.1.3. Skip-Gram model maximizes the following objective:

$$\max_{\theta} \sum_{v_i \in E'} \sum_{v_c \in \mathcal{N}(v_i)} \log p(v_c|v_i; \theta) \tag{15}$$

Finally, the updated embeddings for the DaaSKG entities in E', as well as their proximity degrees are used in the composition of drone services, and the search of relevant

skyway segments, as explained in §5.2 and §5.3. Such a look-ahead approach ensures a configurable delivery plan, as long as the DaaSKG embeddings are incrementally updated.

5.5 Running example

Consider a delivery request from a source to a target station (light coral ellipses in Fig. 5). The example shows that some stations could not be involved in the delivery mission for different reasons. For example, the stations rs_1, rs_5 and rs_{10} (red ellipses) are excluded due to their unavailability or limited charging capacities; while rs_7 (orange ellipse), despite its under-utilization by other flying drones, is excluded due to its incompatible recharging pads, compared to those on the source, intermediate (yellow ellipses $rs_2, rs_3, rs_6, rs_8, rs_9$), and target stations. Following the same logic, some drones (e.g., $\{d_8, d_9, d_{10}\}$ attending rs_1; $\{d_{17}, d_{18}\}$ attending rs_{10}; and $\{d_{20}, d_{21}, d_{22}\}$ attending rs_7) are excluded from the composition process, either because they are involved in other delivery missions, or due to their limited delivery capacities (e.g., payload) and QoS (e.g., delivery cost and time, reputation).

By applying the DaaSKG embedding step (see Section 5.1), the best recharging stations are expected to have close distances and offer similar recharging capacities (e.g., pad type, voltage). In Fig. 5, the intermediate stations $\{rs_2, rs_3, rs_6, rs_8, rs_9\}$ belong to the same vector sub-space of the source and target stations, thanks to their common features (e.g., $padtype : fixed$, $power : 10Kv$). Therefore, they will form several candidate paths, like depicted in Fig. 5 ($P1 : \{DS, rs_2, rs_6, rs_8, DT\}$, $P2 : \{DS, rs_3, rs_6, rs_9, DT\}$). To traverse one of the above paths, several factors are considered as mentioned in Algorithm 2 (i.e. drones' range and payload, segments' distance, stations' waiting and charging times). As shown in Table 3, the delivery mission could be achieved by several drone formations. For example, three low-capacity drones ($\{d_1, d_2, d_3\}$, $\{d_1, d_5, d_6\}$, or $\{d_2, d_5, d_8\}$) may cooperate, but with a low coverage of customers' preferences (penalized score in $[0.751, 0.68]$). A better delivery plan could be obtained by combining d_3 and d_4 services. However, the best case is when a consumer's delivery request is satisfied by a single drone (d_9), that offers high-quality and low-cost delivery services ($score = 0.88$). This latter case avoids the swarming situations and minimizes the waiting time at each station.

6 Complexity analysis

We conducted a theoretical complexity study of the DaaSKG embedding model, the DaaS composition algorithm, and the skyway path search method.

- *DaaSKG embedding:* the complexity of this pre-processing step depends on the cost of metapath-guided random walks, which take $\mathcal{O}(e^2)$, and the Skip-Gram phase with a complexity order of $\mathcal{O}(n * m)$. Here, e and m denote respectively the numbers of edges and nodes in the DaaSKG, whereas m is the Skip-Gram hidden layer dimension. Another factor of embedding complexity concerns the transition probability, which is computed for each pair of nodes (e.g., station-station, drone-drone, drone-station). It takes $\mathcal{O}(p.|T|)$, with l layers, $|\mathcal{V}|$ d-dimensional vectors, $|T|$ types of neighbor nodes, and p meta-paths with their different sizes.
- *Proximity-aware DaaS composition:* this step's complexity depends on the vector space size ($|\mathcal{V}|$), which consists of the numbers of candidate services ($|\mathcal{E}_s|$), drones ($|\mathcal{E}_d|$) and recharging stations $|\mathcal{E}_r|$, in addition to the delivery query $|\mathcal{DR}|$. In the worst case, $|\mathcal{E}_r|$ stations will be processed to extract candidate drone services. For each recharging station, the algorithm checks if the list of drones covers all (or part of) the requested services. The cost of this operation is $\mathcal{O}(|\mathcal{E}_r|.|\mathcal{E}_d|)$. Since for each drone in the recharging station, the delivery constraints are evaluated, this process will also be repeated for each of the services involved in the DaaS composition. Hence, the complexity of this algorithm is $\mathcal{O}(|\mathcal{E}_r|.|\mathcal{E}_d|.|\mathcal{DR}|)$, or briefly $\mathcal{O}(|\mathcal{V}|.|\mathcal{DR}|)$.
- *Skyway path selection:* this step's cost mainly depends on the skyway network size, i.e. number of segments, as well as the number of drone services composed for the delivery task. The skyway segments' information are evaluated against drones' flight capacities to check these latter's ability to traverse each segment. This operation takes $\mathcal{O}(|\mathcal{E}_r|.\frac{|\mathcal{E}_r|-1}{2}.|\mathcal{E}_d|)$. The whole routine is repeated for each candidate path. Therefore, the whole complexity is in $\mathcal{O}(|\mathcal{E}_r|^2.|\mathcal{E}_d|)$.

7 Experimental study

We implemented our approach (DaaSKG embedding, DaaS composition, and skyway path search) using Python programming language on PyTorch IDE. NetworkX python library, , as well as other tools (PyTorch Geometric, Torch-scatter, Torch-sparse) were used to construct and process the skyway network's topology, in which each node is a (recharging) station. The skyway network is also part of the constructed DaaSKG. We used $\langle [S]^+ \rangle$, $\langle [D]^+ \rangle$ and $\langle D\text{-}S\text{-}D \rangle$ as the guiding meta-paths of the generated random walks.

7.1 Dataset description and preprocessing

The data used in the experiments are from a real drone dataset, which contains flight logs from 30 autonomous outdoor drones. The dataset encompasses comprehensive attributes and metrics associated with drones' flight operations. Primary features include drones' battery voltage, speed, flight duration, altitude, coordinates, power consumption, energy consumption, and payload. Based on these raw metrics, several information are derived, like the total energy consumption of each flight, the time required to recharge, the remaining energy, etc.

Because of the small number of drones and to conduct our experiments in alignment with the literature and real-world UAV services, we augmented this dataset with additional drone/station entities. For example, Zipline typically uses 30–50 stations on average in each operational country (Africa and recently U.S.). While Amazon Prime Air delivers small packages (under 5 pounds) using a fleet that is likely under 100 drones currently. Based on that, the generated data (see Table 4) for 80 drone services and 60 stations (five pads per station) include drones' flight capacities (e.g., payload, battery, speed, recharging time, flight range), skyway segments' data (e.g., distances), recharging stations' resources (e.g., types of recharging pads), and user requests (e.g., package weight, sources, and destinations). To compute a drone's total delivery time and cost, we assume that some variables are fixed (e.g., drone speed, time to fully recharge a drone), while others were randomly generated (e.g., skyway segment's distance). Each segment is characterized by the cost and the distance. Other useful data depend on the input delivery requests, the drones' capacity (e.g., flight duration) and the stations' occupancy (waiting and charging time).

In terms of pre-processing, the data undergoes several transformational steps. Distances between sky segments were computed, then, normalized and rescaled to ensure that they are within a realistic range. The dataset is further segmented based on each drone's status, either in-flight or stationed. Moreover, relationships between DaaSKG entities are meticulously defined to transform the data into heterogeneous graph-based structures that is suitable to.

7.2 Compared methods

DaaSKG experiments were conducted on Google Colab platform. To prove the efficiency of applying KGE models in our work, we compared our solution and its incremental variant DaaSKG++ with two game-theoretic approaches [8], namely NCG-CI and NCG-PB.

- **DaaSKG++.** Unlike DaaSKG which computes drones and stations embeddings from scratch, DaaSKG++ incorporates incremental learning by capturing the dynamic updates in the sky network, such as drones changes (e.g., failures, battery level) and stations updates (e.g., high congestion, pads unavailability). DaaSKG++ aims to improve scalability and reduce computational overhead that may be caused by rerunning the whole embedding process. By incrementally updating the drones and stations embeddings, DaaSKG++ is supposed to rapidly locate the new vector subspace containing high-proximity stations and drone services, resulting in a more efficient search of delivery path (i.e. sequence of stations) and selection of drones formation.
- **NCG-CI** (Non-Cooperative Game with Complete Information) serves as a traditional approach, where all players have complete information about the sky network, and make decisions based on the real-time status of recharging stations and competing players. This approach provides an exact solution by exhaustively considering all possible compositions of drone services.
- **NCG-PB** (Prediction-based Non-Cooperative Game) approach aims at handling uncertain and dynamic traffic congestion at different stations. Unlike NCG-CI which assumes a complete knowledge of the drone-based delivery system, NCG-PB predicts congestion levels at intermediate stations. It also handles delivery queries by considering competing drone services and their expected arrival time (a specific time interval) at a specific charging station.

7.3 Results

7.3.1 Distribution of DaaSKG entities

The visualization depicted in Figs. 6 and 7 offers an insightful representation of the recharging stations' and

Table 4 Dataset and experiments variables

Variable	Value
# drones	[10, 80]
# recharging stations	[10, 60]
# recharging pads	[50, 300]
# DaaS	[250, 400]
# skyway segments	[14, 266]
Full recharging time	$\approx$ 60 minutes
Drone speed	65 km/h
Max weight per package	5 kg
# packages per request	8
# generated requests	[5, 10]
Threshold	0.8

drone services' embeddings, based on their inherent features. The projection of these embeddings in the vector space reveals different patterns of similarity or divergence between them.

The learned representations helped excluding, not only the stations with high occupancy and waiting time, but also the drone services with low capacity, thus avoiding the swarming situations. That is also attributed to the correct arrangement of high-proximity stations and drones into the same vector subspace, as previously mentioned in Fig. 2.

7.3.2 Experiments on the composition quality

The goal of these experiments is to study the impact of different factors (numbers of drones and stations, payload) on the delivery cost and the traveled distance (see Fig. 8 and Fig. 9).

Delivery cost. Figure 8a indicates that, For DaaSKG method, there is a gradual increase in the delivery cost as the number of drones increases. This suggests that the complexity or resource needs of the delivery missions increase, leading to higher costs. On the other hand, DaaSKG++ exhibits lower delivery costs compared to DaaSKG, with a different increase rate. Taking the case of 10 and 20 drones, the delivery cost for DaaSKG++ is slightly lower than that of DaaSKG. However, when the number of drones reaches 30, the costs for the two methods converge. This differential behavior might indicate that, while DaaSKG++ is more efficient for smaller drone fleet and change rates, its advantage diminishes as the number of drones increases. Regarding NCG methods, the two variants produced higher delivery costs compared to DaaSKG and DaaSKG++, but with a gradual increase as the number of drones rises (case of [20–30] drones). The delivery cost produced by NCG-PB method is affected by its prediction accuracy, especially with more drones, which causes potential inefficiencies in the resource allocation or the routing strategy.

We also recorded the delivery cost for different payloads (see Fig. 8b). For all the approaches, there is an evident trend that as the payload size increases, so does the delivery cost. That is expected, as larger payloads make delivery missions more complex (caused by drones' capacity and formation), lengthy (caused by charging needs), and energy-consuming. However, it is noticeable that for all the test cases, DaaSKG++ approach consistently results in a lower delivery cost compared to DaaSKG method. This means that for tasks involving large payloads, DaaSKG++ might be the most cost-efficient choice, thanks to its capacity to incrementally capture and handle the DaaSKG updates (e.g., stations availability). Although NCG-CI exhibits a similar behavior to DaaSKG++, its predictive variant has produced the worst delivery plans. That could be noticed from the gap with DaaSKG++, NCG-CI, and even DaaSKG. Indeed, NCG-PB's major goal is to deal with traffic congestion. That becomes challenging with large payloads, which require multiple drones that need to be routed to different stations.

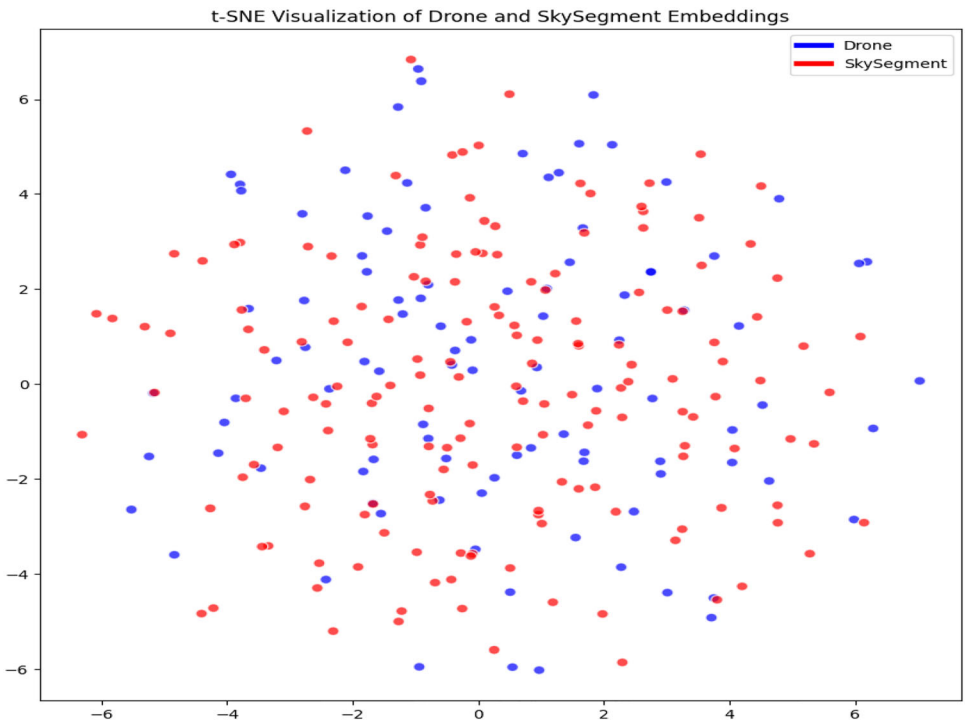

Fig. 6 t-SNE visualization of drones' and segments' embeddings

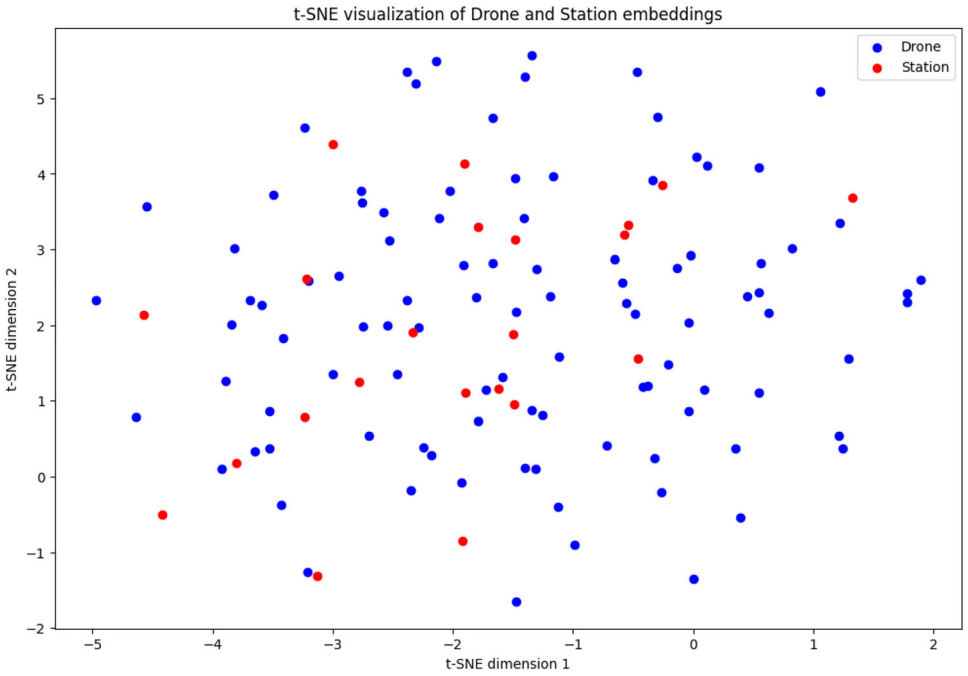

Fig. 7 t-SNE visualization of drones and stations' embeddings

The slightly higher delivery cost by DaaSKG method, compared to NCG-CI and DaaSKG++ (see Fig. 8b), could be explained by its focus on the proximity between the entities of the sky network. These proximity measures are mainly guided by meta-paths and relevant contextual information, which help decide if drone services and stations will be mapped closer in the embedding space. Since this logic relies on common features between entities like drones and stations (e.g., type of pad and voltage), it might target the drones with high similarity in charging features but with low payload capacity. Consequently, multiple drones with limited capabilities will be involved in the DaaS composition, which increases the delivery cost. In contrast, the lower delivery cost of DaaSKG++ is explained by the reduced number of combined drone services. In fact, this incremental variant (unlike DaaSKG) implements metapath2vec++ principles, mainly negative sampling to enhance node separation in the vector space [53]. DaaSKG++ not only takes into consideration similar entities, but also has the capacity to learn the differentiation of sky network entities by identifying which entities are unrelated (e.g., functionally different drone services and stations). This mix of contextual information (i.e., neighbor and negative sampling) has produced richer vector representations compared to the DaaSKG method, in which the negative sampling is not incorporated (i.e. only real relationships are processed), and only neighborhood and metapath-based factual relations have been processed.

Delivery distance. Regardless of the tested approach, the distance traveled rises noticeably as we increase the number of drones (see Fig. 9a). All the methods feature analogous trends in their distance metrics against the drones number, with a slightly shorter distance for DaaSKG methods (case of 30 drones). In fact, an additional number of drones, coupled with their different flight capacities, leads to multiple drone formations which have a direct impact on the generated skyway path and the selected stations. As with the payload size comparison (see Fig. 8b), the differential in travel distances between DaaSKG and DaaSKG++ remains relatively stable, indicating no intrinsic differences in their routing logic.

We also studied the effect of varying the payload on the traveled distance. For all compared methods (Fig. 9b), the distances traveled increase with a larger payload size. This implies that delivering larger payloads might require additional drones with different formations. Consequently, extended distances will be traversed for recharging purposes. Also, the distances traveled are quite proximate when the payload is small (case of 250), with NCG variants, typically showcasing marginally elevated values. However, the distances recorded for NCG-CI and NCG-PB methods for larger payloads (case of 500 and 750) indicate a less optimal routing strategy leading to longer distances traveled and potentially higher costs. Regarding our approach, the disparity in delivery distances between the two algorithms (DaaSKG and DaaSKG++) remains relatively invariant across different payload sizes. This could be attributed to the reduced number of intermediate stations (charging purpose) due to the high flight range and battery capacity of the DaaS formation selected by both methods, regardless of the payload capacity. This latter feature, in contrast, has directly affected the delivery cost as explained

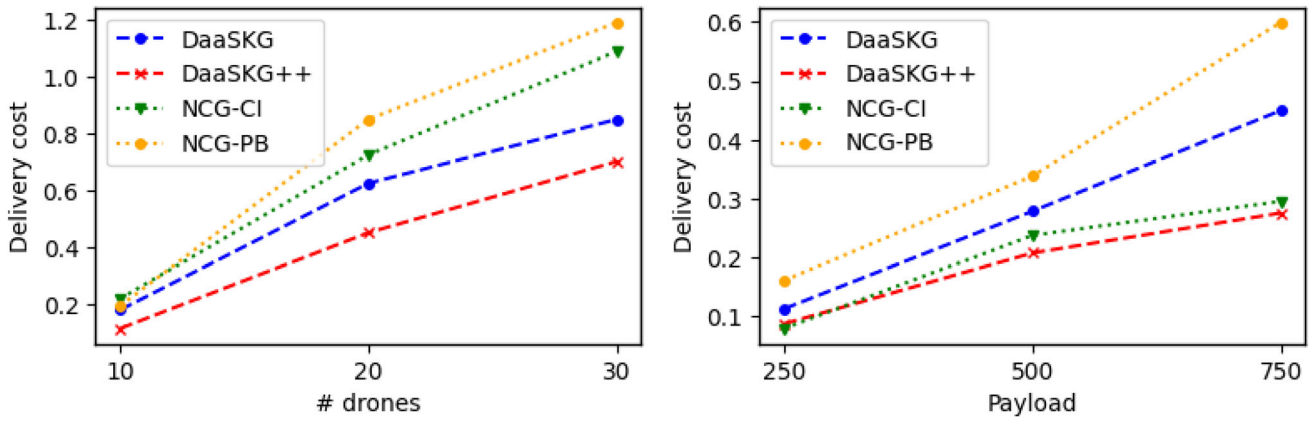

Fig. 8 Impact of the drones number and the payload on the delivery cost

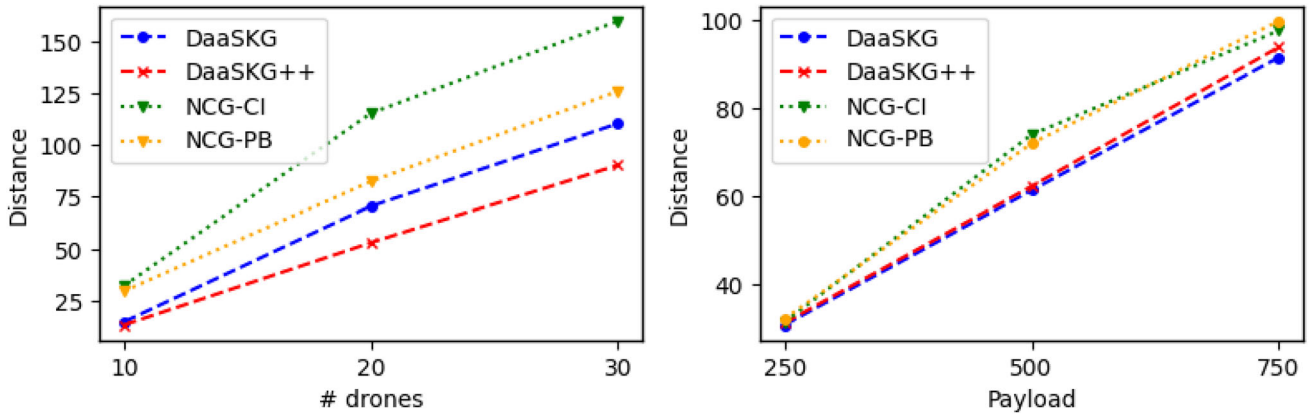

Fig. 9 Impact of the drones number and the payload on the delivery distance

in Fig. 8b. The gap between DaaSKG and NCG approaches could be indicative of systematic variations in the adopted routing strategies or the embeddings engendered by the metapath-based models, culminating in a foreseeable difference in DaaSKG and NCG operational efficacy. Overall, Fig. 9b highlights the challenges of efficiently routing larger payloads, with DaaSKG method showing promising efficiency, regardless of the payload size.

Delivery time. We recorded the average delivery times across varying numbers of stations (see Fig. 10a) and drones (see Fig. 10b). For all the test cases, DaaSKG++ is the best, followed by DaaSKG and NCG-PB. These approaches are associated with shorter delivery times for small sky networks (e.g., [50–100] stations). However, as the number of stations increases (case of [200–300]), there is a slight upward trend in delivery time, indicating that the complexity of the delivery task escalates. NCG-CI aims to approximate near-optimal solutions with reduced computational burden, yet its path generation phase may be affected by prediction inaccuracies and uncertainties in network dynamics. Although NCG-PB exhibits slightly longer delivery times compared to our approach, it maintains competitive performance across varying numbers of stations. In contrast, the reduced delivery times produced by our approach are attributed to the shortest skyway paths, as recorded in Fig. 9. We also notice, from Fig. 10a a decrease in the delivery time (case of [50–80] drones). That was expected for all the approaches, especially for our case, thanks to the embedding step. In fact, a higher number of drones means additional DaaS alternatives with eventually better flight and delivery capacities. While the former (e.g., flight range, battery capacity) helped reduce the number of intermediate stations and consequently the waiting/charging times, the latter capacity (e.g., payload) helped achieve the delivery mission with a smaller number of drones, thus reducing the swarming situations.

Overall, the capacity of capturing complex relationships and dependencies, as well as the encoding of multi-hop and

metapath-guided sequences, allowed projecting this kind of knowledge into rich vector representations. In addition, the dense information in the generated embeddings facilitated these latter's arrangement into distinct vector sub-spaces (e.g., stations in specific flight regions, drones with specific mission types). This enabled the composition method to locate nearby stations and exploit their high proximity with candidate drone services. Combining context awareness (higher-order neighborhood) with metapath-guided sampling was effective and improved the embedding accuracy. As a result, the exploration of the embedding space was more efficient thanks to the precise mapping of drone services and stations, which facilitated their capabilities matching. Unlike DaaSKG and DaaSKG++, NCG-CI and NCG-PB lack context-awareness and cannot capture semantic and broader relational information (i.e., drones and stations neighborhood). NCG-CI and NCG-PB are unable to integrate high-dimensional features, and they are limited to a simple modeling of competitive interactions between players, as well as basic relationships, due to the use of simplified payoff structures. Moreover, agents in NCG-CI and NCG-PG methods operate individually to maximize their utility over the efficiency of the global solution (i.e., whole delivery plan). Furthermore, their decisions are independent and lead to sub-optimal solutions. Consequently, the matching of their goals lacks precision as it is based only on similar payoffs.

7.3.3 Computation time analysis

Observing the previously recorded metrics (delivery cost, delivery time, and traveled distance), we recognize a direct correlation with the density (number of segments) and configuration of the skyway network. Specifically, with the increase in the number of drones and stations, we inevitably encounter an augmented time complexity in the DaaS filtering (see Fig. 11a) and skyway paths' search (see Fig. 11b). This complexity arises from the multiplicity of drone services (case of [60–80]) and sky segments (case of 250 and 300 stations), leading to variances in the aforementioned metrics. As the skyway network grows and becomes denser, one would expect more diversified pathways that might be traversed by drones, given the expanded recharging options (e.g., [250–300]). It is also to note that, with a fixed-size sky network and a varied number of drones (see Fig. 11a), DaaSKG method outperformed its incremental version, as the amount of changes in the stations' states is lower than the updates in drone states (in-flight or stationed).

The second tests were conducted with a varied number of pads ([50,300:50]) and a fixed number of drones (see Fig. 11b). DaaSKG++ outperforms all the approaches, except for the case of [50,100] stations, where NCG-CI benefits from a reduced overhead thanks to its nearly optimal solutions. DaaSKG++ incremental nature proved its suitability to dynamic and large drone networks (case of [250–300]), in which only the captured changes are handled. In fact, the embedding-based exploration methodology entails the systematic structuring and modeling of drones and charging stations via knowledge graphs. These entities' vectorized forms, as well as their proximity degrees, facilitated a meticulous analysis of each node's features and latent information, which also reduced the cost of refining candidate delivery plans. DaaSKG method features a higher computation time, compared to DaaSKG++, because its embedding step maps all the entities to the vector space, regardless of the captured changes. With the negative sampling and the incremental

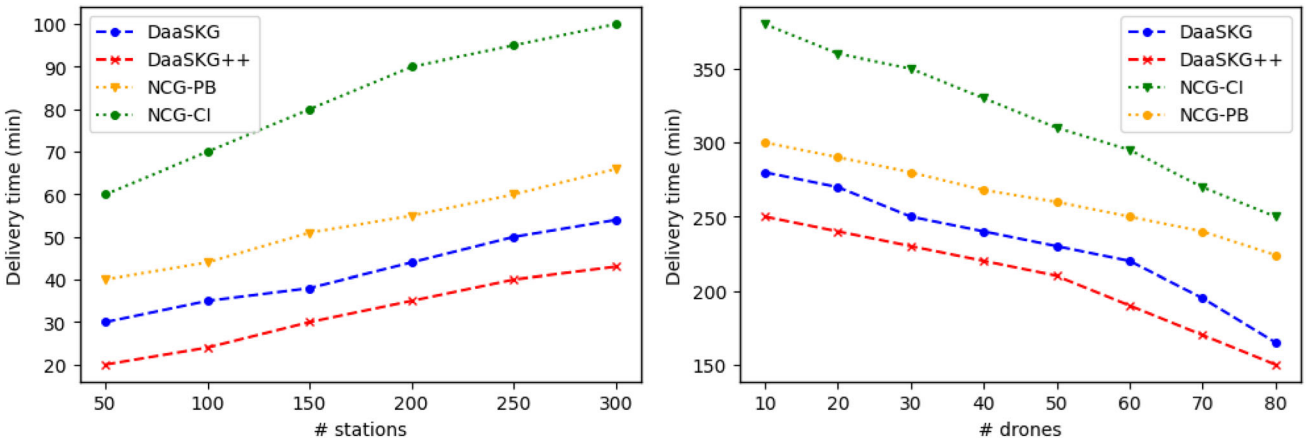

Fig. 10 Delivery times with varying numbers of drones and stations

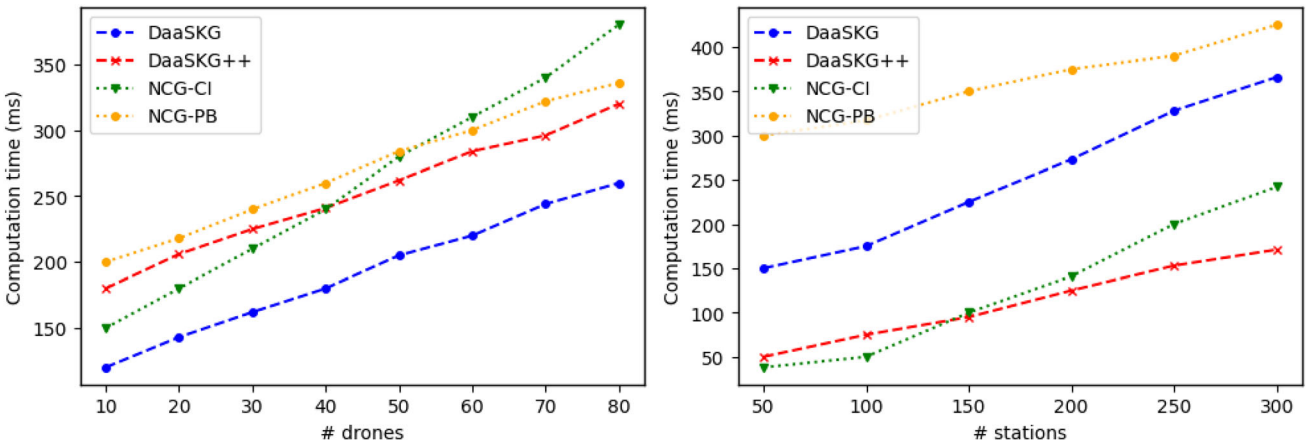

Fig. 11 Computation times with varying numbers of drones and stations

processing logic, DaaSKG++ is scalable and its produced embeddings retain their quality without requiring a costly re-computation based on unnecessary triples.

As for NCG-PB, it exhibits prolonged computation times and constrained scalability due to its exhaustive exploration of all feasible DaaS compositions. NCG-CI performed well only with 50-to-100 stations, which confirms its suitability for small environments. However, we noticed a significant increase in its computation time when the sky network enlarges (e.g., more than 150 stations). Although NCG-CI was faster than its predictive variant (NCG-PB), its poor scaling was expected with the increasing number of players (e.g., drones). This non-cooperative game model requires an iterative and frequent recalculation to reach equilibrium, which becomes computationally intensive in large UAV networks.

7.3.4 Parameters sensitivity analysis

These experiments aim to study the impact of the parameters used in the metapath-guided model on the quality of vector embeddings (representing DaaSKG entities) and, consequently, on the solution quality. In fact, a good embedding quality reflects an accurate projection of DaaSKG entities with similar features as close as possible in the vector space. Therefore, each subspace can be seen as a cluster of similar vectorized entities. We give as example an embedding subspace of the drones having similar flight capacities (e.g., battery capacity, flight speed and range, payload) or an embedding subspace of the drones and stations with similar charging features (e.g., voltage, pad type, charging power). Based on the above, tests were carried out by varying the number of walks in [1,5], the walks length in [2,4], the window size in [2,5], and the embedding dimension in [64,512] (see Fig. 12).

Number of walks. The number of walks per DaaSKG entity (e.g., drone service, station) helps determine the node sequences generated for that entity. Fig. 12a shows that the distribution of DaaSKG entities in the vector space can be influenced by the number of walks, as in the case of [2–4] walks. The embedding quality was improved with 5 walks per node, which underlines the importance of guided sampling in the metapath-based embedding step. By gravitating towards a more intensive sampling of DaaSKG sequences, the DaaS composition and sky segments' search steps will gain from a more semantic and a richer sampling. In fact, the higher quality of distribution with 5 walks results in a more authentic representation of drone services and charging stations, which is justified by the reduced costs and shorter delivery paths (see Fig. 8).

Increasing the number of walks helped the embedding model capture a rich mix of DaaSKG entities and their typed relations based on the predefined meta-paths (e.g., *pad-station-drone* metapath). This resulted in an improved semantic nuance of DaaSKG entities, which ensured discovering significant patterns from the generated sequences. Therefore, the high quality of generated embeddings will better guide the arrangement of DaaSKG entities in distinct vector sub-spaces, w.r.t. their closeness (i.e. common features), which improves the process of selecting drone services and locating closely connected stations. In contrast, a lower number of walks prevents the embedding model from exploring higher-order relationships, as the node sequences are limited to a small number of context entities. Consequently, the vector subspaces will be small and very specific, reducing the embeddings quality and making the search for suitable drones and stations a difficult task.

Walk length. In the DaaSKG, a longer walk length allows to capture a longer sequence of relationships, unlike short walk lengths which mainly target local relationships (e.g., drone-station triples). Short walks, as depicted in

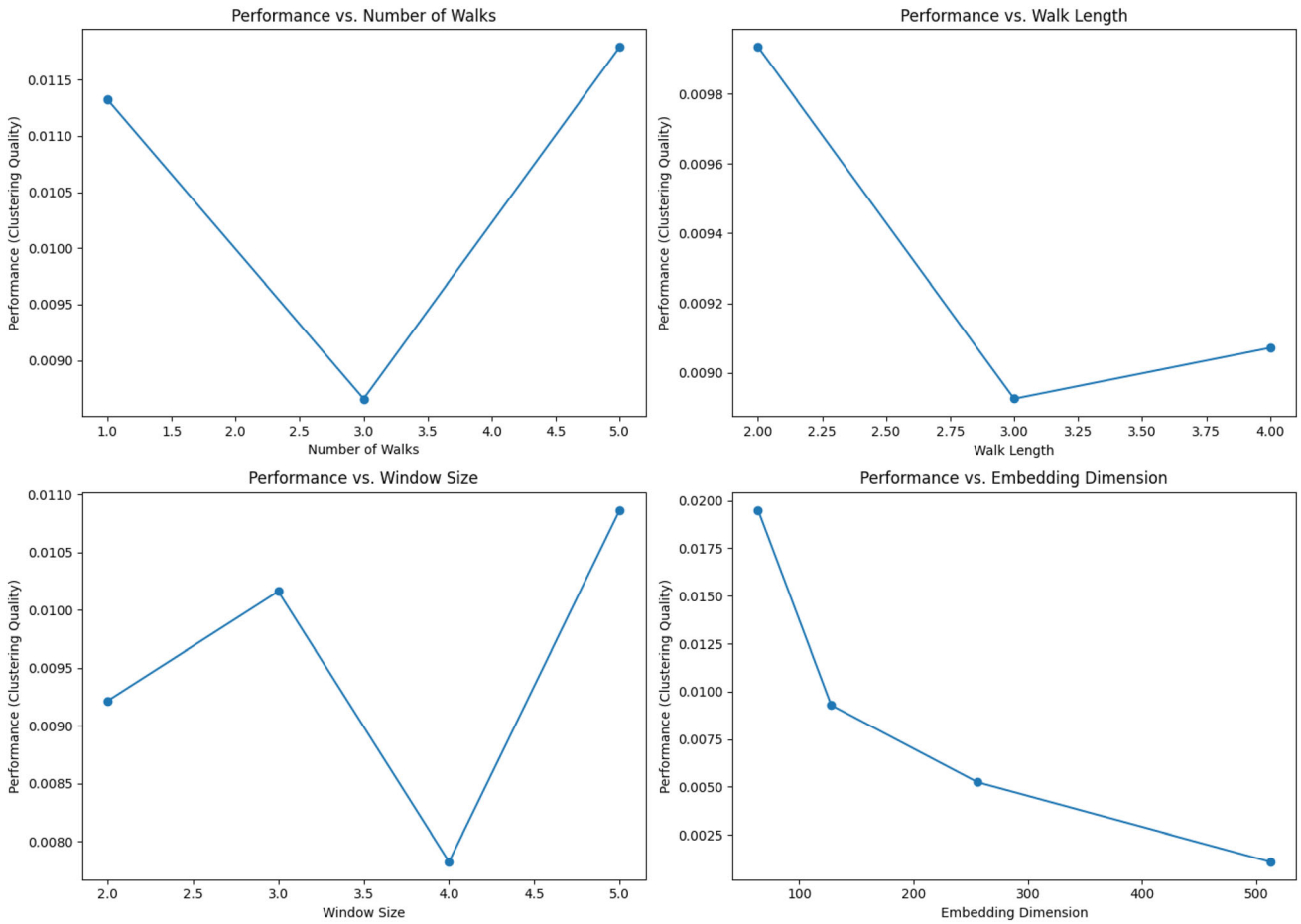

Fig. 12 Impact of embedding parameters on the DaaSKG vector representation

Fig. 12b, resulted in small-size (i.e. overly specific and fragmented) distinct vector subspaces. Also, the embedding sub-spaces might only capture DaaSKG entities with specific features, without taking into account other common features. We give as examples drone entities for which only drone characteristics are taken into consideration, while ignoring other useful information such as traversed sky segments and visited stations. Indeed, that limited the ability to encode long chains of relationships and diverse dependencies (i.e. typed relations). As a result, the generated embeddings missed out relevant patterns (i.e. important connections) with neighbor entities.

Looking at the results with longer walks (e.g., *length=4*), these latter allowed the vector embeddings to incorporate information from multiple sequences in the DaaSKG, thus, capturing auxiliary but relevant relationships between entities. For example, complex chains of dependencies between stations translate into flight meta-paths, while long-range sequences of drones and stations express charging patterns. The higher walk lengths in Fig. 12b resulted in a lower loss, as DaaSKG nodes shared several common features, which guaranteed their accurate arrangement in the same clusters of vectors (i.e. embedding subspaces). In Fig. 12b, the consistent performance trajectory across varying walk lengths epitomizes the composition method's resilience to this embedding hyper-parameter. However, the ephemeral decrement at a walk length of 3 could be attributed to the flight dataset's intricate topological contours. While the walk length does not drastically modulate the embedding quality, the results suggest that the DaaSKG structure remains fairly steadfast across distinct walk lengths.

Embedding dimension. The embedding dimensions varied in these tests (between 64 and 512) indicate the length of DaaSKG entities' vector representations in the embedding space. For example, a high dimension of 256 translates into more encoded information, captured complex structures and relation patterns within the DaaSKG, such as flight patterns (i.e. specific delivery paths), charging patterns (i.e. core/central stations), etc. The inverse relationship between the embedding dimension and the clustering quality (see Fig. 12d) underscores a pivotal observation: a higher dimension does not invariably translate to a superior representation of the DaaSKG

content. As we increase the embedding dimensions, the model may increasingly fall to over-fitting or noise encapsulation. An embedding dimension of 64, in particular, appears to furnish a more streamlined and efficient representation of the DaaSKG entities than its more extensive counterpart, 512. In fact, a moderate dimension might produce a more nuanced vector representation of DaaSKG entities, which increases the accuracy of their arrangement in the embedding space.

Increasing the embedding dimension has integrated more information into the vector representations, which helped capture significant differences between DaaSKG entities, leading to more distinct embedding subspaces. However, when the DaaSKG entities are projected into a low-dimensional vector space (e.g., 64–128), the embedding model falls into under-fitting. Indeed, drones and stations representations in this case will consist of a limited number of features, while other essential relationships and unique features of related entities (e.g., charging pads) may be omitted. As a result, the generated embeddings will be overly similar for DaaSKG entities that are expected to be heterogeneous and projected far from each other in different vector subspaces (i.e. blurring distinctions between vector subspaces). In other words, the results for low embedding dimensions reflect a poor vector representation caused by the high loss (see Fig. 12). That means the metapath-guided embedding model has failed to differentiate between DaaSKG entities (e.g., two drones with specific flight capacity, or stations with unique charging features), resulting in poorer representations. Consequently, low embedding dimensions have mapped low-similarity DaaSKG entities closer in the embedding space, which had a negative impact on the quality of DaaS compositions.

Window size. This metapath2vec parameter was varied (see Fig. 12c) to study the impact of capturing broader contextual information around each DaaSKG entity on its learned representation and, consequently, on its chance to participate in the delivery plan. For example, the windows size for a charging station's related data, also called node context, might include the station's features, its charging pads, the nearby stations, in addition to the visiting drones; whereas a drone service's context is structured as this entity's neighborhood, including the similar drone services, the previously visited stations, etc. By increasing the window size, the learned representations for DaaSKG entities were improved, as more neighboring entities are included. That had a positive impact on the projection of the nodes in the embedding space. However, low window sizes have affected the accuracy of the vectorized forms. That is understandable because the context (i.e. neighborhood) around each DaaSKG entity was restricted and limited to first-order connections. The embedding quality was significantly affected with smaller window sizes, because essential broader information and relations have been missed. As a result, the original DaaSKG structure was not preserved and the entities mapping to the vector space was not accurate. With this over-fragmentation of the DaaSKG embedding space, the approach fails to locate the appropriate drone services and charging stations, due to their inaccurate proximity degrees. To sum-up, the deviations in some test cases confirm the importance of fine-tuned parameters in order for the embedding model to capture the optimal neighborhood scope for each DaaSKG entity.

8 Conclusion and future work

In this paper, we addressed the problem of drone services' composition for the purpose of delivery. To generate the appropriate package delivery plan, which consists of a combined set of drone services and their traversed skyway segments, we first modeled the drones' environment as an information network, namely *DaaSKG*. This latter was, thereafter, translated into a low-dimensional vector space using metapath2vec embedding model, and explored based on a subgraph-aware measure of nodes' proximity degrees. We also defined a composition algorithm that, based on the proximity between the DaaSKG entities (e.g., drones, charging stations), extracts the drones that fulfill at best the delivery mission in a reduced time and cost. The experimental results proved the utility of exploring the DaaSKG vector representations, rather than processing raw features of the graph-like sky network, which accurately and efficiently drove the drone services' composition process.

The present approach does not treat other delivery patterns (e.g., simultaneous pickup and delivery, swarm formations). These latter influence the selection strategy of the stations involved in the delivery missions and, especially, in the recharging tasks. Moreover, our approach does not treat the case when multiple drones need charging resources or approach the same station for the recharging purpose. That will be addressed by covering additional flight scenarios, handling the dynamics of charging stations, and routing the low-battery drones to the nearest stations, without violating the delivery constraints (e.g., time and cost).

Also, current approaches, including the present work, ignore the uncertainty factors, such as the missing information on stations' availability and charging resources, the weather conditions, and drones' failures. Such uncertainty must be continuously handled during the DaaSKG embedding and exploration. Therefore, it is important to encode additional information (e.g., spatio-temporal information, truth degrees of the DaaSKG relations) in

order to preserve uncertainty. To ensure an uncertainty-aware construction and embedding [17], the DaaSKG will be augmented with probabilistic facts and will provide confidence scores, along with each contextual relation between the DaaSKG entities.

Acknowledgements This research project was funded by the Deanship of Scientific Research, Princess Nourah bint Abdulrahman University, through the Program of Research Project Funding After Publication, grant No (44-PRFA-P-91).

Funding The authors have not disclosed any funding.

Data availability Enquiries about data availability should be directed to the authors.

Declarations

Competing Interests The authors have not disclosed any competing interests.

References

1. Li, X., Lee, G.J.X., Yuen, K.F.: Consumer acceptance of urban drone delivery: the role of perceived anthropomorphic characteristics. Cities **148**, 104867 (2024)
2. Shahzaad, B., Bouguettaya, A., Mistry, S., Neiat, A.G.: "Composing drone-as-a-service (daas) for delivery," In: 2019 IEEE International Conference on Web Services (ICWS). IEEE, pp. 28–32 (2019)
3. Bouguettaya, A., Singh, M., Huhns, M., Sheng, Q.Z., Dong, H., Yu, Q., Neiat, A.G., Mistry, S., Benatallah, B., Medjahed, B., et al.: A service computing manifesto: the next 10 years. Commun. of the ACM **60**(4), 64–72 (2017)
4. Shahzaad, B., Bouguettaya, A., Mistry, S., Neiat, A.G.: Resilient composition of drone services for delivery. Futur. Gener. Comput. Syst. **115**, 335–350 (2021)
5. Dorling, K., Heinrichs, J., Messier, G.G., Magierowski, S.: "Vehicle routing problems for drone delivery,." IEEE Trans. Syst. Man, and Cybern. **47**, 70–85 (2016)
6. Park, S., Zhang, L., Chakraborty, S.: "Design space exploration of drone infrastructure for large-scale delivery services," In: 35th International Conference on Computer-Aided Design, pp. 1–7 (2016)
7. Neiat, A.G.: Constraint-aware drone-as-a-service composition. In: 17th International conference on service-oriented computing, ICSOC 2019, 11895, pp. 369–382. Springer, Cham (2019)
8. Shahzaad, B., Bouguettaya, A., Mistry, S.: "A game-theoretic drone-as-a-service composition for delivery," In: 2020 IEEE International Conference on Web Services (ICWS). IEEE, pp. 449–453 (2020)
9. Alkouz, B., Bouguettaya, A., Mistry, S.: "Swarm-based drone-as-a-service (sdaas) for delivery," In: 2020 IEEE International Conference on Web Services (ICWS). IEEE, pp. 441–448 (2020)
10. Chu, L., Li, X., Xu, J., Neiat, A.G., Liu, X.: "A holistic service provision strategy for drone-as-a-service in mec-based uav delivery," In: International Conference on Web Services (ICWS). IEEE, pp. 669–674 (2021)
11. Hamdi, A., Salim, F.D., Kim, D.Y., Neiat, A.G., Bouguettaya, A.: Drone-as-a-service composition under uncertainty. IEEE Trans. on Serv. Comput. **15**(5), 2685–2698 (2021)
12. Alkouz, B., Bouguettaya, A.: "Provider-centric allocation of drone swarm services," In: 2021 IEEE International Conference on Web Services (ICWS). IEEE, pp. 230–239 (2021)
13. Alkouz, B., Abusafia, A., Lakhdari, A., Bouguettaya, A.: "In-flight energy-driven composition of drone swarm services," IEEE Transactions on Services Computing, (2022)
14. Alkouz, B., Bouguettaya, A., Lakhdari, A.: "Density-based pruning of drone swarm services," In: 2022 IEEE International Conference on Web Services (ICWS). IEEE, pp. 302–311 (2022)
15. Alkouz, B., Bouguettaya, A., Lakhdari, A.: "Failure-sentient composition for swarm-based drone services," In: International Conference on Web Services (ICWS). IEEE, pp. 493–503 (2023)
16. Shahzaad, B., Alkouz, B., Janszen, J., Bouguettaya, A.: Optimizing drone delivery in smart cities. Internet Comput. **27**, 32 (2023)
17. Cao, J., Fang, J., Meng, Z., Liang, S.: Knowledge graph embedding: a survey from the perspective of representation spaces. ACM Comput. Surv. **56**, 1–42 (2023)
18. Sorbelli, F.B.: Uav-based delivery systems: a systematic review, current trends, and research challenges. Auton. Transp. Syst. **1**, 1–40 (2024)
19. Murray, C.C., Chu, A.G.: The flying sidekick traveling salesman problem: optimization of drone-assisted parcel delivery. Transp. Res. Part C: Emerg. Technol. **54**, 86–109 (2015)
20. Tu, P.A., Dat, N.T., Dung, P.Q.: "Traveling salesman problem with multiple drones," In: Proceeding of the 9th International Symposium on Information and Communication Technology, pp. 46–53 (2018)
21. Ferrandez, S.M., Harbison, T., Weber, T., Sturges, R., Rich, R.: Optimization of a truck-drone in tandem delivery network using k-means and genetic algorithm. J. Ind. Eng. Manag. (JIEM) **9**(2), 374–388 (2016)
22. Marinelli, M., Caggiani, L., Ottomanelli, M., Dell'Orco, M.: En route truck-drone parcel delivery for optimal vehicle routing strategies. IET Intell. Transp. Syst. **12**(4), 253–261 (2017)
23. Agatz, N., Bouman, P., Schmidt, M.: Optimization approaches for the traveling salesman problem with drone. Transp. Sci. **52**(4), 965–981 (2018)
24. Ha, Q.M., Deville, Y., Pham, Q.D., Ha, M.H.: On the min-cost traveling salesman problem with drone. Transp. Res. Part C: Emerg. Technol. **86**, 597–621 (2018)
25. Yurek, E.E., Ozmutlu, H.C.: A decomposition-based iterative optimization algorithm for traveling salesman problem with drone. Transp. Res. Part C: Emerg. Technol. **91**, 249–262 (2018)
26. Bouman, P., Agatz, N., Schmidt, M.: Dynamic programming approaches for the traveling salesman problem with drone. Networks **72**(4), 528–542 (2018)
27. Kim, S., Moon, I.: Traveling salesman problem with a drone station. IEEE Trans. Syst., Man, and Cybern. Syst. **49**(1), 42–52 (2018)
28. Jeong, H.Y., Song, B.D., Lee, S.: Truck-drone hybrid delivery routing: Payload-energy dependency and no-fly zones. Int. J. Prod. Econ. **214**, 220–233 (2019)
29. Kitjacharoenchai, P., Ventresca, M., Moshref-Javadi, M., Lee, S., Tanchoco, J.M., Brunese, P.A.: Multiple traveling salesman problem with drones: Mathematical model and heuristic approach. Comput. Ind. Eng. **129**, 14–30 (2019)
30. Zhao, L., Bi, X., Dong, Z., Xiao, N., Zhao, A.: Robust traveling salesman problem with drone: balancing risk and makespan in contactless delivery. Int. Trans. Oper. Res. **31**(1), 167–191 (2024)
31. Wang, X., Poikonen, S., Golden, B.: The vehicle routing problem with drones: Several worst-case results. Optim. Lett. **11**(4), 679–697 (2017)
32. Poikonen, S., Wang, X., Golden, B.: The vehicle routing problem with drones: extended models and connections. Networks **70**(1), 34–43 (2017)

33. Pugliese, L.D.P., Guerriero, F.: "Last-mile deliveries by using drones and classical vehicles," In: International Conference on Optimization and Decision Science. Springer, pp. 557–565 (2017)
34. Campbell, J.F., Sweeney, D., Zhang, J.: "Strategic design for delivery with trucks and drones," Supply Chain Analytics Report SCMA (04 2017), (2017)
35. Schermer, D., Moeini, M., Wendt, O.: A variable neighborhood search algorithm for solving the vehicle routing problem with drones. Technical Report Technische Universität Kaiserslautern, Tech. Rep. (2018)
36. Ham, A.M.: Integrated scheduling of m-truck, m-drone, and m-depot constrained by time-window, drop-pickup, and m-visit using constraint programming. Transp. Res. Part C: Emerg. Technol. **91**, 1–14 (2018)
37. Ulmer, M.W., Thomas, B.W.: Same-day delivery with heterogeneous fleets of drones and vehicles. Networks **72**(4), 475–505 (2018)
38. Karak, A., Abdelghany, K.: The hybrid vehicle-drone routing problem for pick-up and delivery services. Transp. Res. Part C: Emerg. Technol. **102**, 427–449 (2019)
39. Wang, Z., Sheu, J.-B.: Vehicle routing problem with drones. Transp. Res. part B **122**, 350–364 (2019)
40. Poikonen, S., Golden, B.: Multi-visit drone routing problem. Comput. Operat. Res. **113**, 104802 (2020)
41. Kitjacharoenchai, P., Min, B.-C., Lee, S.: Two echelon vehicle routing problem with drones in last mile delivery. Int. J. Prod. Econ. **225**, 107598 (2020)
42. Jiang, J., Dai, Y., Yang, F., Ma, Z.: A multi-visit flexible-docking vehicle routing problem with drones for simultaneous pickup and delivery services. Eur. J. Operat. Res. **312**(1), 125–137 (2024)
43. Chu, J.C., Shui, C., Lin, K.-H.: Optimization of trucks and drones in tandem delivery network with drone trajectory planning. Comput. Ind. Eng. **189**, 110000 (2024)
44. Hong, F., Wu, G., Luo, Q., Liu, H., Fang, X., Pedrycz, W.: "Logistics in the sky: A two-phase optimization approach for the drone package pickup and delivery system," IEEE Transactions on Intelligent Transportation Systems, (2023)
45. Jana, S., Mandal, P.S.: Approximation algorithms for drone delivery scheduling with a fixed number of drones. Theor. Comput. Sci. **991**, 114442 (2024)
46. Yanpirat, N., Silva, D.F., Smith, A.E.: Sustainable last mile parcel delivery and return service using drones. Eng. Appl. Artif. Intell. **124**, 106631 (2023)
47. Wen, X., Wu, G., Li, S., Wang, L.: Ensemble multi-objective optimization approach for heterogeneous drone delivery problem. Exp. Syst. Appl. **249**, 123472 (2024)
48. Murray, C.C., Raj, R.: The multiple flying sidekicks traveling salesman problem: parcel delivery with multiple drones. Transp. Res. Part C **110**, 368–398 (2020)
49. Yılmaz, C., Cengiz, E., Kahraman, H.T.: A new evolutionary optimization algorithm with hybrid guidance mechanism for truck-multi drone delivery system. Exp. Syst. Appl. **245**, 123115 (2024)
50. Lee, W., Shahzaad, B., Alkouz, B., Bouguettaya, A.: "Reactive composition of uav delivery services in urban environments," Preprint at arXiv:2404.18363, (2024)
51. Chen, X., Jia, S., Xiang, Y.: A review: knowledge reasoning over knowledge graph. Exp. Sys. Appl. **141**, 112948 (2019)
52. Zhou, J., Liu, L., Wei, W., Fan, J.: Network representation learning: from preprocessing, feature extraction to node embedding. ACM Comput. Surv. (CSUR) **55**(2), 1–35 (2022)
53. Dong, Y., Chawla, N.V., Swami, A.: "metapath2vec: Scalable representation learning for heterogeneous networks," In: 23rd ACM SIGKDD International Conference on knowledge discovery and data mining, pp. 135–144 (2017)
54. Choudhury, S., Agarwal, K., Purohit, S., Zhang, B., Pirrung, M., Smith, W., Thomas, M.: "Nous: Construction and querying of dynamic knowledge graphs," In: 2017 IEEE 33rd International Conference on Data Engineering (ICDE). IEEE, pp. 1563–1565 (2017)

Mokhtar Sellami received his PhD in computer science in 2016 from Tunis El Manar University, Tunisia. He is currently a member of RIADI research laboratory and an Assistant professor with the University of Jendouba, Tunisia. His research interests include modern data management systems, formal concept Analysis, data security & privacy, cloud computing. He has also published several research papers in well-known conferences and journals.

Haithem Mezni received the PhD in computer science from Manouba University in 2014. He is an associate professor with Jendouba University and a member of SMART Research Laboratory, Tunisia. His current research interests include service and cloud computing, and recommender systems. His publication record includes articles in peer-reviewed journals such as IEEE TKDE, TSC, Grid Computing, DKE, FGCS, SoCo, IJAR, etc. He has served as a journal guest editor and a member of several international conferences.

Hela Elmannai received her PhD degree in Information Technology from SUPCOM in Tunisia. She is currently an assistant professor with the College of Computer and Information Sciences, Princess Nourah bint Abdulrahman University, Saudi Arabia. Her research interests include artificial intelligence, networking, blockchain and engineering applications.

Reem Alkanhel received the PhD degree in information technology (networks and communication systems) from Plymouth University, United Kingdom, in 2019. She is currently an assistant professor at the college of Computer and Information Sciences, Princess Nourah bint Abdulrahman University, Saudi Arabia. Her current research interests include communication systems, networking, Internet of Things, Software-Defined Networking, and Information Security.